

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет систем управления и робототехники

Лабораторная работа №D
по дисциплине
Теория автоматического управления

Вариант 10

Студент:

Журавлев Михаил Романович

Преподаватель:

Пащенко Артем Витальевич

Санкт-Петербург
2026

Содержание

Содержание	2
1. Задание 1	3
1.1. Проверка стабилизируемости	3
1.2. Интегральный функционал качества	4
1.3. Синтез регуляторов	6
1.4. Моделирование	7
1.5. Вывод	11
2. Задание 2	11
2.1. Проверка обнаруживаемости	12
2.2. Синтез наблюдателя	12
2.3. Моделирование	15
2.4. Вывод	22
3. Задание 3	23
3.1. Проверка стабилизируемости и обнаруживаемости	24
3.2. Синтез регулятора и наблюдателя	24
3.3. Моделирование	26
3.4. Вывод	28
4. Общий вывод	29
5. Приложение	30
5.1. MATLAB-скрипт для задания 1	30
5.2. MATLAB-скрипт для задания 2	38
5.3. MATLAB-скрипт для задания 3	46

Задание 1

Рассматривается система

$$\dot{x} = Ax + Bu, \quad x(0) = x_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix},$$

где

$$A = \begin{bmatrix} 5 & 6 & 3 \\ -6 & -7 & -6 \\ 6 & 6 & 5 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

Требуется синтезировать регулятор

$$u = Kx$$

из условия минимума интегрального квадратичного функционала качества

$$J = \int_0^\infty (x^T(t)Qx(t) + u^T(t)Ru(t)) dt.$$

Проверка стабилизируемости

Перед синтезом регулятора проверим, можно ли стабилизировать систему обратной связью по состоянию. Матрица управляемости равна

$$U = [B \ AB \ A^2B] = \begin{bmatrix} 1 & 5 & 7 \\ 0 & -6 & -24 \\ 0 & 6 & 24 \end{bmatrix}, \quad \text{rank}(U) = 2.$$

Следовательно, система не является полностью управляемой. Поэтому дополнительно проверяется условие стабилизируемости: все неуправляемые моды должны быть устойчивыми. Собственные числа матрицы A :

$$\sigma(A) = \{2 + 3i, 2 - 3i, -1\}.$$

Характеристический полином, по которому получен этот спектр:

$$\chi_{A(\lambda)} = \det(\lambda I - A) = \lambda^3 - 3\lambda^2 + 9\lambda + 13 = (\lambda + 1)(\lambda^2 - 4\lambda + 13).$$

Матрицы Хаутуса

$$\text{rank} [(2 + 3i)I - A \ B] = 3, \quad \text{rank} [(2 - 3i)I - A \ B] = 3.$$

Для устойчивой моды $\lambda = -1$:

$$H(-1) = \begin{bmatrix} -6 & -6 & -3 & 1 \\ 6 & 6 & 6 & 0 \\ -6 & -6 & -6 & 0 \end{bmatrix}, \quad \text{rank} [-1I - A \ B] = 2.$$

Неуправляемой является только мода $\lambda = -1$. Она лежит в левой полуплоскости, поэтому система стабилизируема. Это означает, что регулятор $u = Kx$ может сделать замкнутую систему асимптотически устойчивой, хотя произвольно назначить все три собственных числа нельзя.

Интегральный функционал качества

Интегральный показатель качества имеет вид суммы двух квадратичных интегральных оценок:

$$J = J_x + J_u = \int_0^\infty x^T Q x \, dt + \int_0^\infty u^T R u \, dt.$$

Матрица $Q = Q^T > 0$ задает штраф за отклонение состояния от нуля, а матрица $R = R^T > 0$ задает штраф за управление. При увеличении Q регулятор сильнее подавляет состояние; при увеличении R управление становится более дорогим, поэтому регулятор действует мягче.

Для системы с законом $u = Kx$ оптимальный регулятор находится через матричное уравнение Риккати при $\nu = 1$:

$$A^T P + P A + Q - P B R^{-1} B^T P = 0, \quad K = -R^{-1} B^T P.$$

Так как управление скалярное, R является положительным числом. При $R = 1$ формула регулятора принимает вид $K = -B^T P$, а при $R = \alpha$:

$$K = -\frac{1}{\alpha} B^T P.$$

Решение $P = P^T > 0$ задает квадратичную функцию $V(x) = x^T P x$. После подстановки оптимального управления $u = Kx$ замкнутая система имеет вид

$$\dot{x} = (A + BK)x,$$

а подынтегральное выражение функционала можно записать как

$$x^T Q x + u^T R u = x^T (Q + K^T R K) x.$$

Минимальное значение функционала при начальном состоянии x_0 вычисляется по формуле

$$J_{\min} = x_0^T P x_0.$$

Для сравнения влияния весов выбраны

$$Q = I_3, \quad R = 1, \quad \alpha = 5.$$

Тогда рассматриваются четыре пары:

$$(Q, R), \quad (\alpha Q, R), \quad (Q, \alpha R), \quad (\alpha Q, \alpha R).$$

Для каждой пары решается свое уравнение Риккати. В частности, для базовой пары $(Q, R) = (I_3, 1)$:

$$A^T P_1 + P_1 A + I_3 - P_1 B B^T P_1 = 0, \quad K_1 = -B^T P_1.$$

Для пары $(\alpha Q, R)$:

$$A^T P_2 + P_2 A + \alpha I_3 - P_2 B B^T P_2 = 0, \quad K_2 = -B^T P_2.$$

Для пары $(Q, \alpha R)$:

$$A^T P_3 + P_3 A + I_3 - \frac{1}{\alpha} P_3 B B^T P_3 = 0, \quad K_3 = -\frac{1}{\alpha} B^T P_3.$$

Для пары $(\alpha Q, \alpha R)$:

$$A^T P_4 + P_4 A + \alpha I_3 - \frac{1}{\alpha} P_4 B B^T P_4 = 0, \quad K_4 = -\frac{1}{\alpha} B^T P_4.$$

Для произвольной строки обратной связи $K = [k_1 \ k_2 \ k_3]$ замкнутая матрица имеет вид

$$A + BK = \begin{bmatrix} 5 + k_1 & 6 + k_2 & 3 + k_3 \\ -6 & -7 & -6 \\ 6 & 6 & 5 \end{bmatrix}.$$

Поскольку неуправляемая мода сохраняется, для всех найденных регуляторов в спектре замкнутой системы остается собственное число -1 . Две остальные моды изменяются выбором весов Q и R .

Так как $x_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, минимальное значение функционала для каждой матрицы P_i равно сумме всех элементов этой матрицы:

$$J_{\min, i} = x_0^T P_i x_0 = [1 \ 1 \ 1] P_i \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \sum_{m=1}^3 \sum_{n=1}^3 p_{i, mn}.$$

Синтез регуляторов

Для пары (Q, R) получено

$$P_1 = \begin{bmatrix} 8.7024141005 & 8.3260987009 & 7.3017545485 \\ 8.3260987009 & 8.8789766614 & 7.7263673688 \\ 7.3017545485 & 7.7263673688 & 10.1221500622 \end{bmatrix},$$

$$K_1 = [-8.7024141005 \quad -8.3260987009 \quad -7.3017545485], \quad J_{\min,1} = 74.4119820606.$$

Спектр замкнутой системы:

$$\sigma(A + BK_1) = \{-2.3512070502 + 3.1667293211i, -2.3512070502 - 3.1667293211i, -1\}.$$

По результатам моделирования на интервале $[0, 12]$:

$$J_{\exp,1}(12) = 74.4602479239.$$

Для пары $(\alpha Q, R)$:

$$P_2 = \begin{bmatrix} 10.4200373857 & 9.1097853389 & 9.0578524441 \\ 9.1097853389 & 11.9411530792 & 11.3205756636 \\ 9.0578524441 & 11.3205756636 & 15.8544484197 \end{bmatrix},$$

$$K_2 = [-10.4200373857 \quad -9.1097853389 \quad -9.0578524441], \quad J_{\min,2} = 97.1920657777.$$

$$\sigma(A + BK_2) = \{-3.2100186928 + 3.5782984795i, -3.2100186928 - 3.5782984795i, -1\}.$$

По результатам моделирования:

$$J_{\exp,2}(12) = 97.2544612021.$$

Для пары $(Q, \alpha R)$:

$$P_3 = \begin{bmatrix} 40.7965234460 & 40.3689266122 & 34.0277624952 \\ 40.3689266122 & 40.9230458733 & 34.4521308370 \\ 34.0277624952 & 34.4521308370 & 43.9836719159 \end{bmatrix},$$

$$K_3 = [-8.1593046892 \quad -8.0737853224 \quad -6.8055524990], \quad J_{\min,3} = 343.4008811240.$$

$$\sigma(A + BK_3) = \{-2.0796523446 + 3.0372609164i, -2.0796523446 - 3.0372609164i, -1\}.$$

По результатам моделирования:

$$J_{\exp,3}(12) = 343.6260769770.$$

Для пары $(\alpha Q, \alpha R)$:

$$P_4 = \begin{bmatrix} 43.5120705023 & 41.6304935046 & 36.5087727426 \\ 41.6304935046 & 44.3948833070 & 38.6318368442 \\ 36.5087727426 & 38.6318368442 & 50.6107503109 \end{bmatrix},$$

$$K_4 = [-8.7024141005 \quad -8.3260987009 \quad -7.3017545485], \quad J_{\min,4} = 372.0599103030.$$

$$\sigma(A + BK_4) = \{-2.3512070502 + 3.1667293211i, -2.3512070502 - 3.1667293211i, -1\}.$$

По результатам моделирования:

$$J_{\text{exp},4}(12) = 372.3012396190.$$

Одновременное умножение Q и R на α не меняет регулятор, потому что относительный вес состояния и управления сохраняется. Поэтому $K_4 = K_1$, а значение функционала масштабируется: $J_{\min,4} = \alpha J_{\min,1}$.

Моделирование

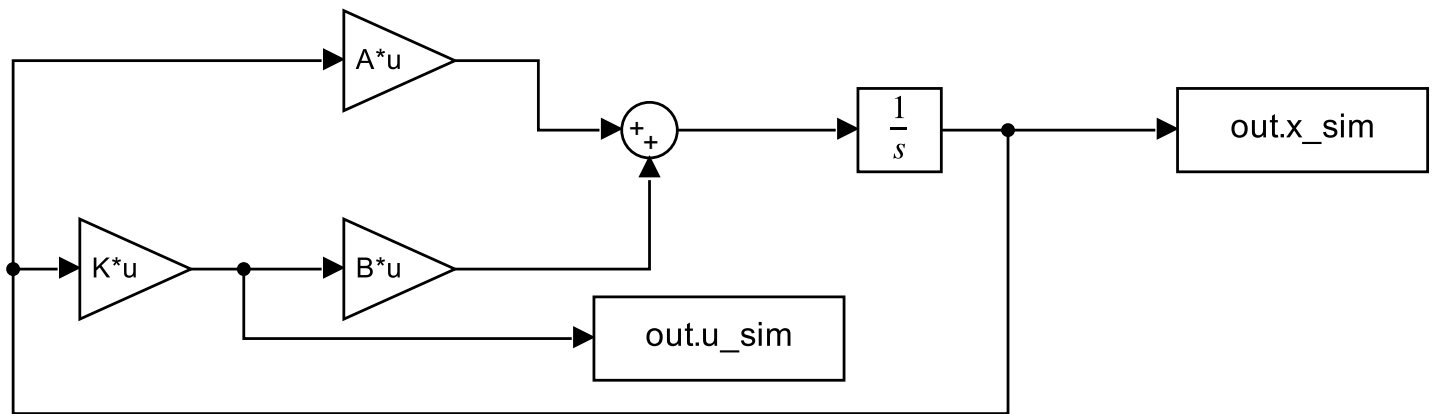


Рисунок 1 — Схема моделирования замкнутой системы

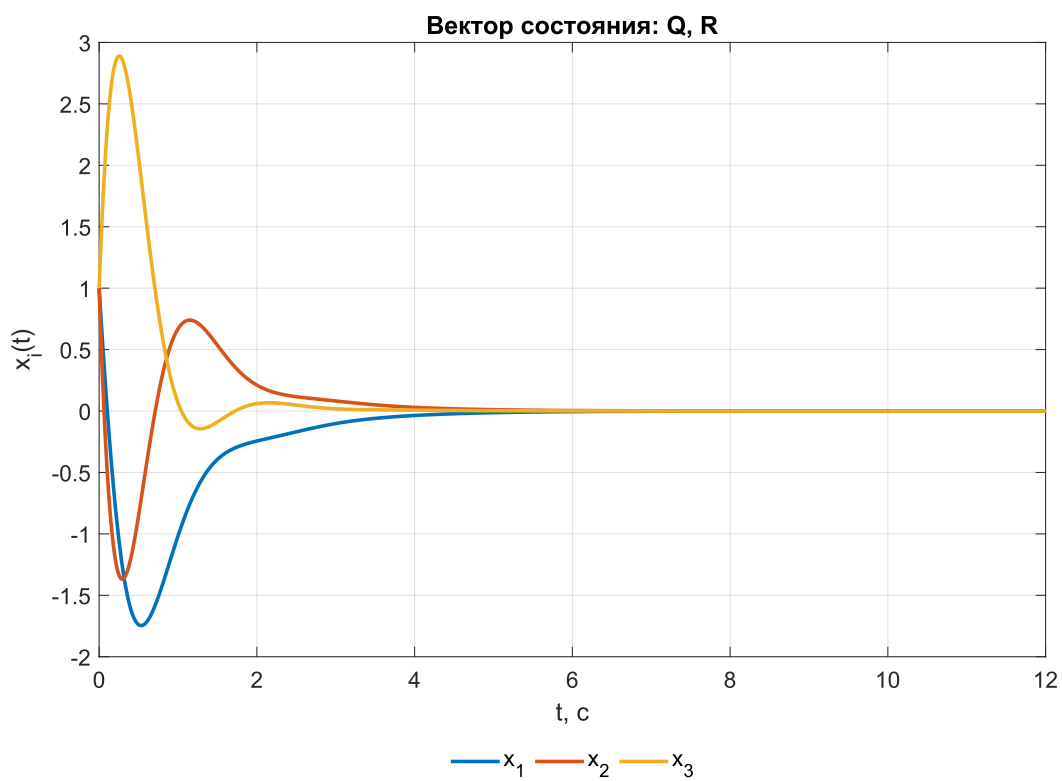


Рисунок 2 — Вектор состояния для пары (Q, R)

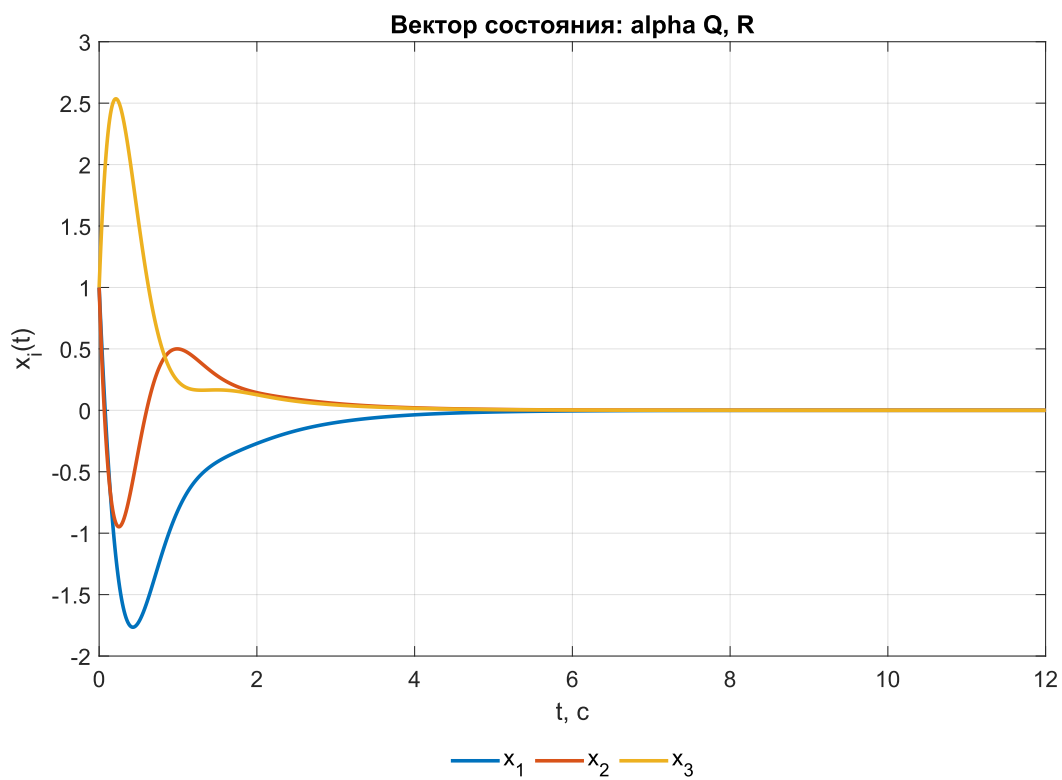


Рисунок 3 — Вектор состояния для пары $(\alpha Q, R)$

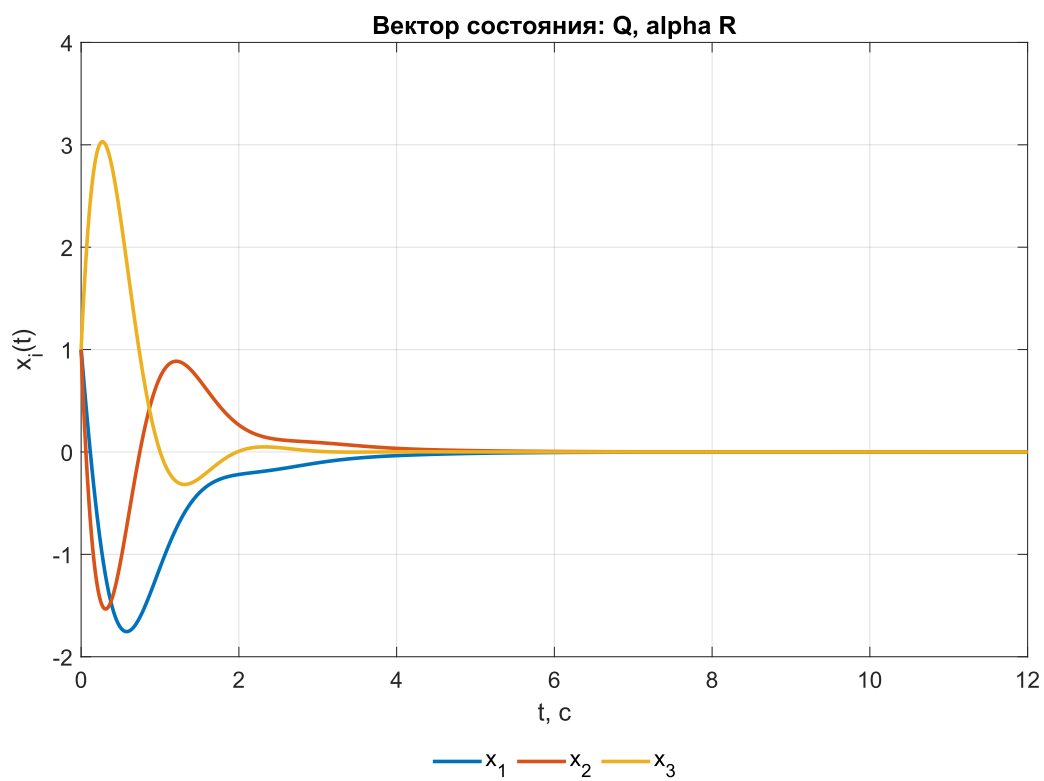


Рисунок 4 — Вектор состояния для пары $(Q, \alpha R)$

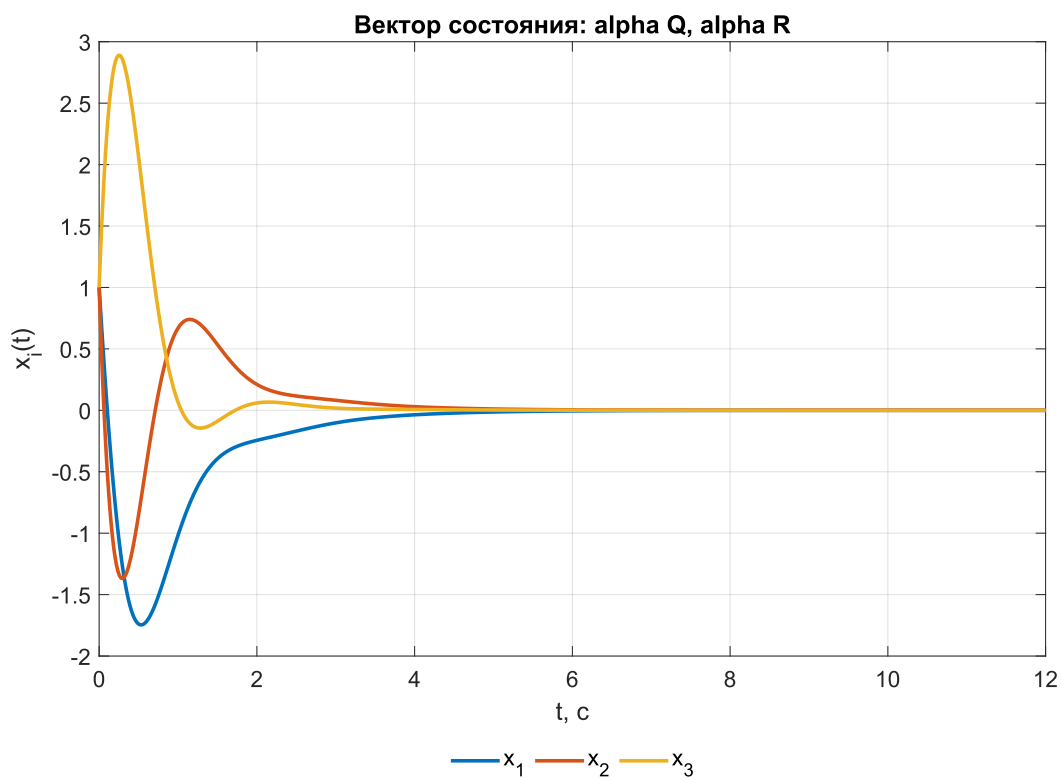


Рисунок 5 — Вектор состояния для пары $(\alpha Q, \alpha R)$

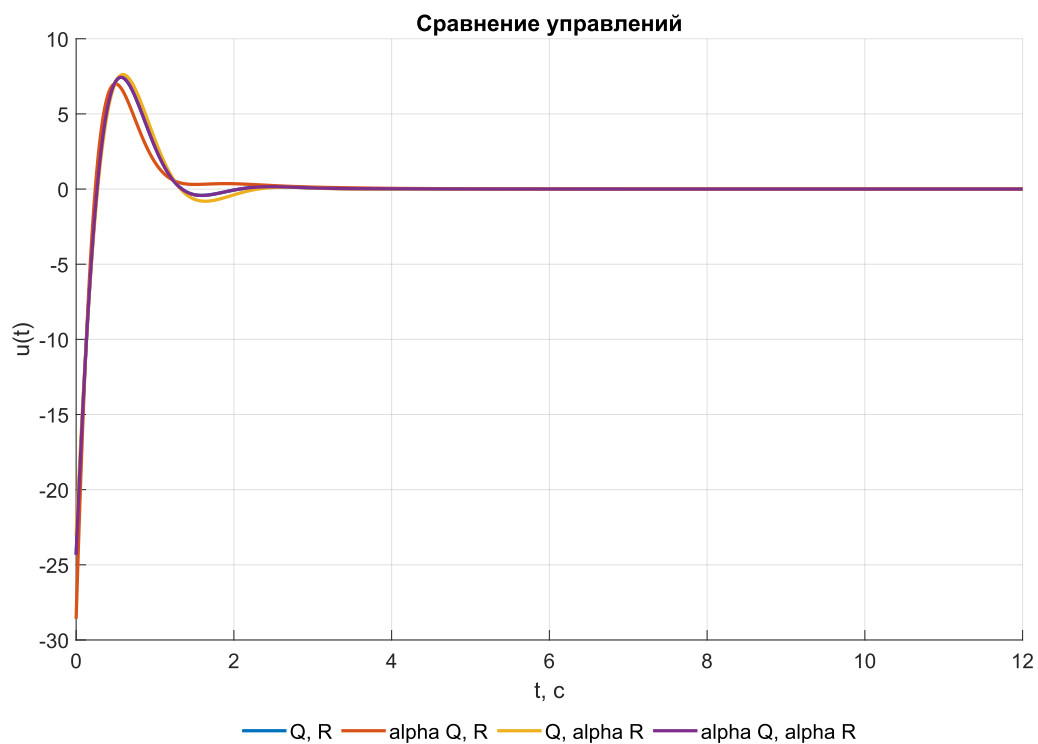


Рисунок 6 — Сравнение сигналов управления $u(t)$

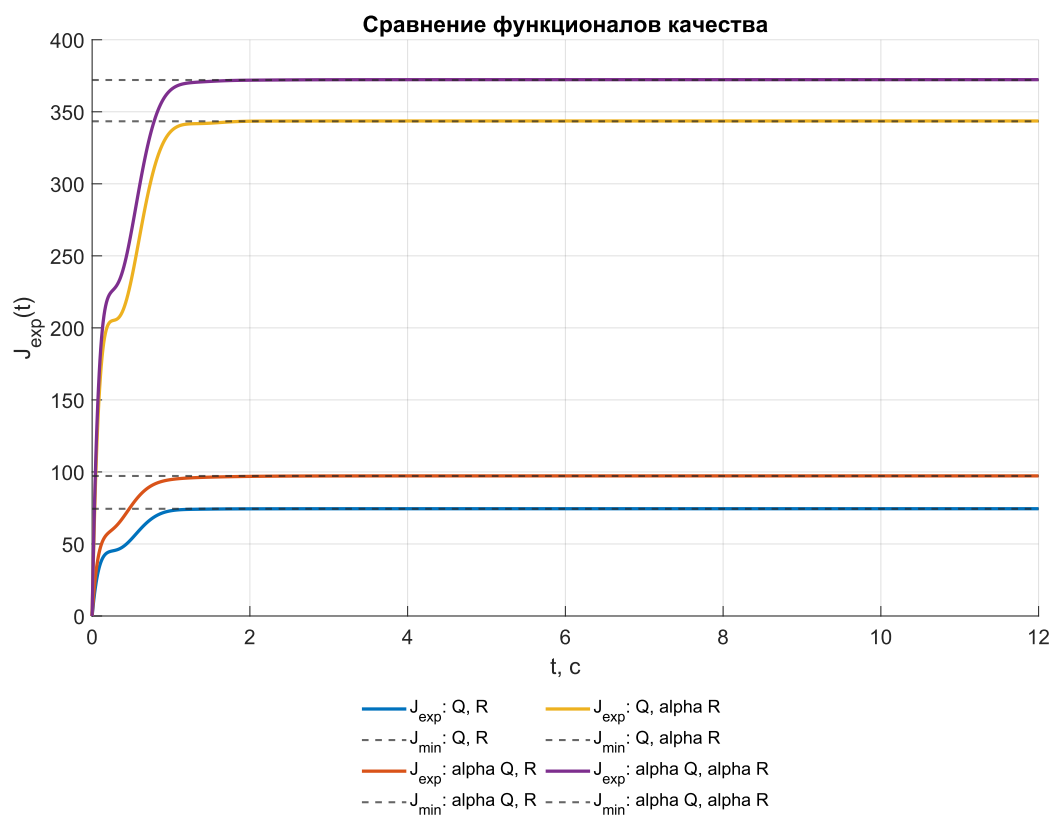


Рисунок 7 — Сравнение $J_{exp}(t)$ и J_{min}

Во всех случаях $J_{\text{exp}(12)}$ близок к соответствующему J_{min} . Небольшое превышение связано с тем, что $J_{\text{exp}(t)}$ вычисляется численным интегрированием сигналов модели на конечном интервале времени.

Вывод

Система не полностью управляема, но стабилизируема, поскольку единственная неуправляемая мода равна -1 и является устойчивой. Увеличение веса Q приводит к более интенсивному управлению и более быстрым управляемым модам замкнутой системы. Увеличение веса R уменьшает усиление регулятора и делает переходный процесс медленнее. Одновременное масштабирование Q и R не меняет матрицу K , но масштабирует значение функционала качества.

Задание 2

Рассматривается система

$$\begin{cases} \dot{x} = Ax + f \\ y = Cx + \xi \end{cases} \quad x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{x}(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

где

$$A = \begin{bmatrix} 35 & 10 & -28 & 17 \\ -22 & -7 & 18 & -12 \\ 56 & 17 & -45 & 27 \\ 34 & 12 & -28 & 17 \end{bmatrix}, \quad C = [7 \ 2 \ -5 \ 3].$$

Так как рассматривается четный вариант, в качестве $f(t)$ и $\xi(t)$ используются случайные гауссовские сигналы с нулевым математическим ожиданием. Вектор $f(t) \in \mathbb{R}^4$ действует в уравнении состояния, а скаляр $\xi(t)$ добавляется к измеряемому выходу.

Наблюдатель состояния задается в виде

$$\dot{\hat{x}} = A\hat{x} + L(C\hat{x} - y).$$

Для невязки

$$e = x - \hat{x}$$

получаем

$$\dot{e} = \dot{x} - \dot{\hat{x}} = Ax + f - A\hat{x} - L(C\hat{x} - y).$$

Так как $y = Cx + \xi$, то

$$C\hat{x} - y = C\hat{x} - Cx - \xi = -Ce - \xi.$$

Следовательно,

$$\dot{e} = (A + LC)e + f + L\xi.$$

Однородная часть динамики ошибки определяется матрицей $A + LC$. Поэтому матрицу L нужно выбрать так, чтобы эта матрица была устойчивой, а влияние случайных сигналов $f(t)$ и $\xi(t)$ на невязку было минимальным в смысле выбранного критерия.

Проверка обнаруживаемости

Матрица наблюдаемости равна

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \\ CA^3 \end{bmatrix} = \begin{bmatrix} 7 & 2 & -5 & 3 \\ 23 & 7 & -19 & 11 \\ -39 & -10 & 29 & -19 \\ -167 & -55 & 139 & -83 \end{bmatrix}, \quad \text{rank}(O) = 4.$$

Следовательно, система полностью наблюдаема, а значит и обнаруживаема. Спектр матрицы A :

$$\sigma(A) = \{+i, -i, +3i, -3i\}.$$

Для всех собственных чисел выполняется условие Хаутуса наблюдаемости:

$$\begin{aligned} \text{rank} \begin{bmatrix} +iI - A \\ C \end{bmatrix} &= 4, & \text{rank} \begin{bmatrix} -iI - A \\ C \end{bmatrix} &= 4, \\ \text{rank} \begin{bmatrix} +3iI - A \\ C \end{bmatrix} &= 4, & \text{rank} \begin{bmatrix} -3iI - A \\ C \end{bmatrix} &= 4. \end{aligned}$$

Синтез наблюдателя

Для синтеза матрицы коррекции используется критерий доверия

$$J = \int_0^\infty (f^{T(t)} Q^{-1} f(t) + \xi^{T(t)} R^{-1} \xi(t)) dt.$$

Матрица $Q = Q^T > 0$ задает интенсивность возмущения в канале состояния, а $R = R^T > 0$ задает интенсивность шума измерения. Чем больше Q относительно R , тем сильнее наблюдатель доверяет измерению и тем активнее корректирует оценку. Чем больше R относительно Q , тем меньше доверие к измерению и тем мягче коррекция.

Матрица L находится через решение уравнения Риккати при $\nu = 1$:

$$AP + PA^T + Q - PC^T R^{-1} CP = 0, \quad L = -PC^T R^{-1}.$$

Выбраны базовые веса

$$Q = I_4, \quad R = 1, \quad \alpha = 5.$$

Рассматриваются четыре пары:

$$(Q, R), \quad (\alpha Q, R), \quad (Q, \alpha R), \quad (\alpha Q, \alpha R).$$

Для пары (Q, R) :

$$P_1 = \begin{bmatrix} 5.3347890859 & -5.0026021987 & 6.0116689097 & 2.5231676192 \\ -5.0026021987 & 11.4491686022 & -5.9938955078 & -6.6297735279 \\ 6.0116689097 & -5.9938955078 & 11.8947254583 & 11.7484472849 \\ 2.5231676192 & -6.6297735279 & 11.7484472849 & 19.3777871834 \end{bmatrix},$$

$$L_1 = \begin{bmatrix} -4.8494775132 \\ 2.0397212313 \\ -5.8656059149 \\ -3.7937514045 \end{bmatrix}.$$

$$\sigma(A + L_1 C) = \{-7.6421831692, -3.7920256535,$$

$$-0.2429579730 + 1.7176261673i, -0.2429579730 - 1.7176261673i\}.$$

По результатам моделирования:

$$\max_t \|e_1(t)\| = 2.4963659842, \quad \|e_1(16)\| = 0.3212022398.$$

Для пары $(\alpha Q, R)$:

$$P_2 = \begin{bmatrix} 6.8823144520 & -8.5821125523 & 6.8311905472 & 3.1242419458 \\ -8.5821125523 & 24.1453424710 & -9.3941075421 & -12.4054051289 \\ 6.8311905472 & -9.3941075421 & 18.7413476027 & 23.5504889349 \\ 3.1242419458 & -12.4054051289 & 23.5504889349 & 41.8104487061 \end{bmatrix},$$

$$L_2 = \begin{bmatrix} -6.2287491608 \\ 2.0297806004 \\ -5.9748475371 \\ -4.7377848064 \end{bmatrix}.$$

$$\sigma(A + L_2 C) = \{-20.2686754154, -3.1673345754,$$

$$-0.2223948337 + 1.7248377218i, -0.2223948337 - 1.7248377218i\}.$$

$$\max_t \|e_2(t)\| = 2.0643662913, \quad \|e_2(16)\| = 0.2186712927.$$

Для пары $(Q, \alpha R)$:

$$P_3 = \begin{bmatrix} 22.0166118190 & -17.9155554512 & 25.9505034512 & 9.8396873240 \\ -17.9155554512 & 38.2766812012 & -22.2981035063 & -23.3445269797 \\ 25.9505034512 & -22.2981035063 & 46.5945287339 & 40.0080138051 \\ 9.8396873240 & -23.3445269797 & 40.0080138051 & 63.6958159618 \end{bmatrix},$$

$$L_3 = \begin{bmatrix} -3.6103433094 \\ 1.4797178328 \\ -4.8217429783 \\ -2.6472272337 \end{bmatrix}.$$

$$\sigma(A + L_3 C) = \{-2.7439873550 + 2.4760309432i, -2.7439873550 - 2.4760309432i, \\ -0.3289797997 + 1.6753999766i, -0.3289797997 - 1.6753999766i\}.$$

$$\max_t \|e_3(t)\| = 4.6243509585, \quad \|e_3(16)\| = 0.6207494446.$$

Для пары $(\alpha Q, \alpha R)$:

$$P_4 = \begin{bmatrix} 26.6739454294 & -25.0130109936 & 30.0583445482 & 12.6158380962 \\ -25.0130109936 & 57.2458430111 & -29.9694775391 & -33.1488676396 \\ 30.0583445482 & -29.9694775391 & 59.4736272916 & 58.7422364243 \\ 12.6158380962 & -33.1488676396 & 58.7422364243 & 96.8889359168 \end{bmatrix},$$

$$L_4 = \begin{bmatrix} -4.8494775132 \\ 2.0397212313 \\ -5.8656059149 \\ -3.7937514045 \end{bmatrix}.$$

$$\sigma(A + L_4 C) = \{-7.6421831692, -3.7920256535, \\ -0.2429579730 + 1.7176261673i, -0.2429579730 - 1.7176261673i\}.$$

$$\max_t \|e_4(t)\| = 2.4956917313, \quad \|e_4(16)\| = 0.3232580138.$$

Одновременное умножение Q и R на α не меняет матрицу L , так как в уравнении Риккати сохраняется относительное соотношение доверия к возмущению состояния и шуму измерения. Поэтому $L_4 = L_1$.

Моделирование

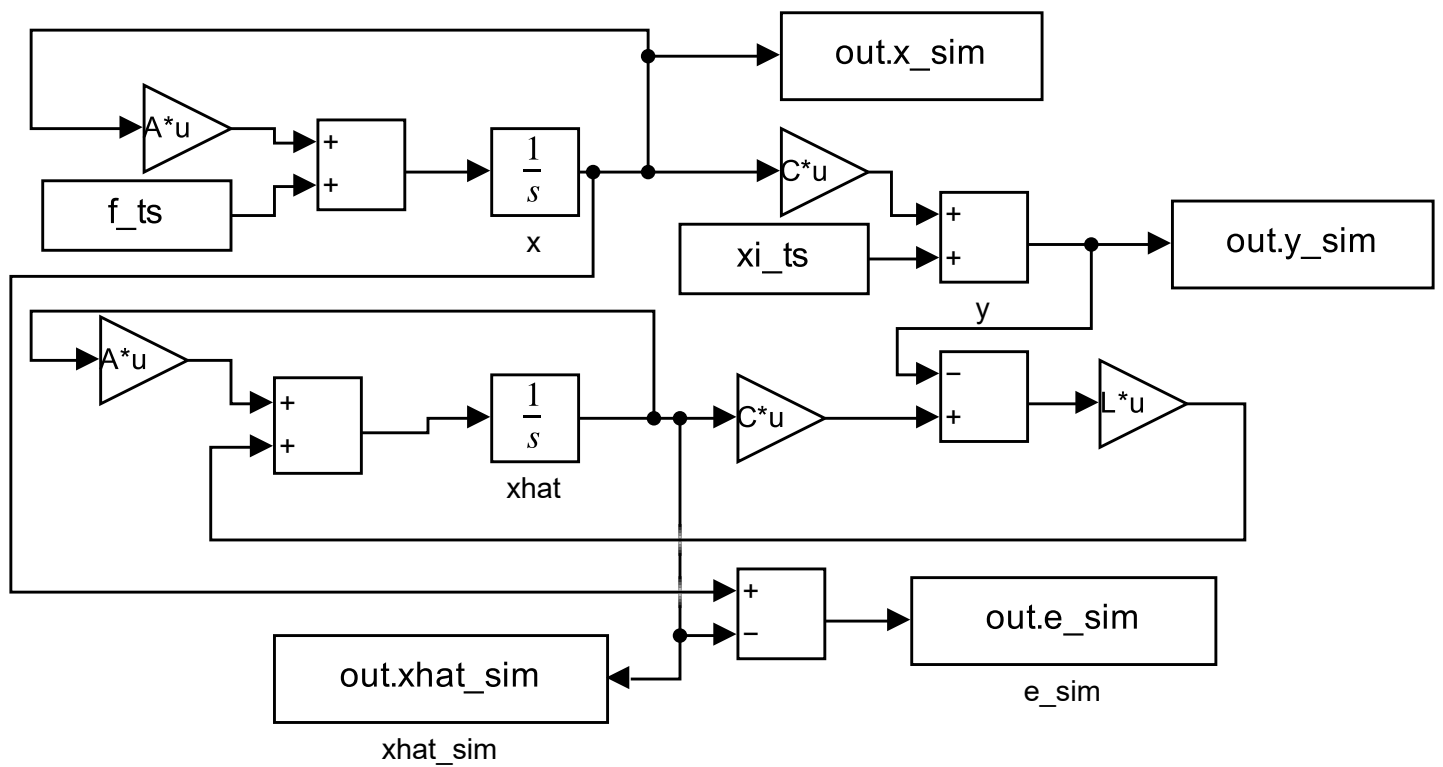


Рисунок 8 — Схема моделирования системы с наблюдателем

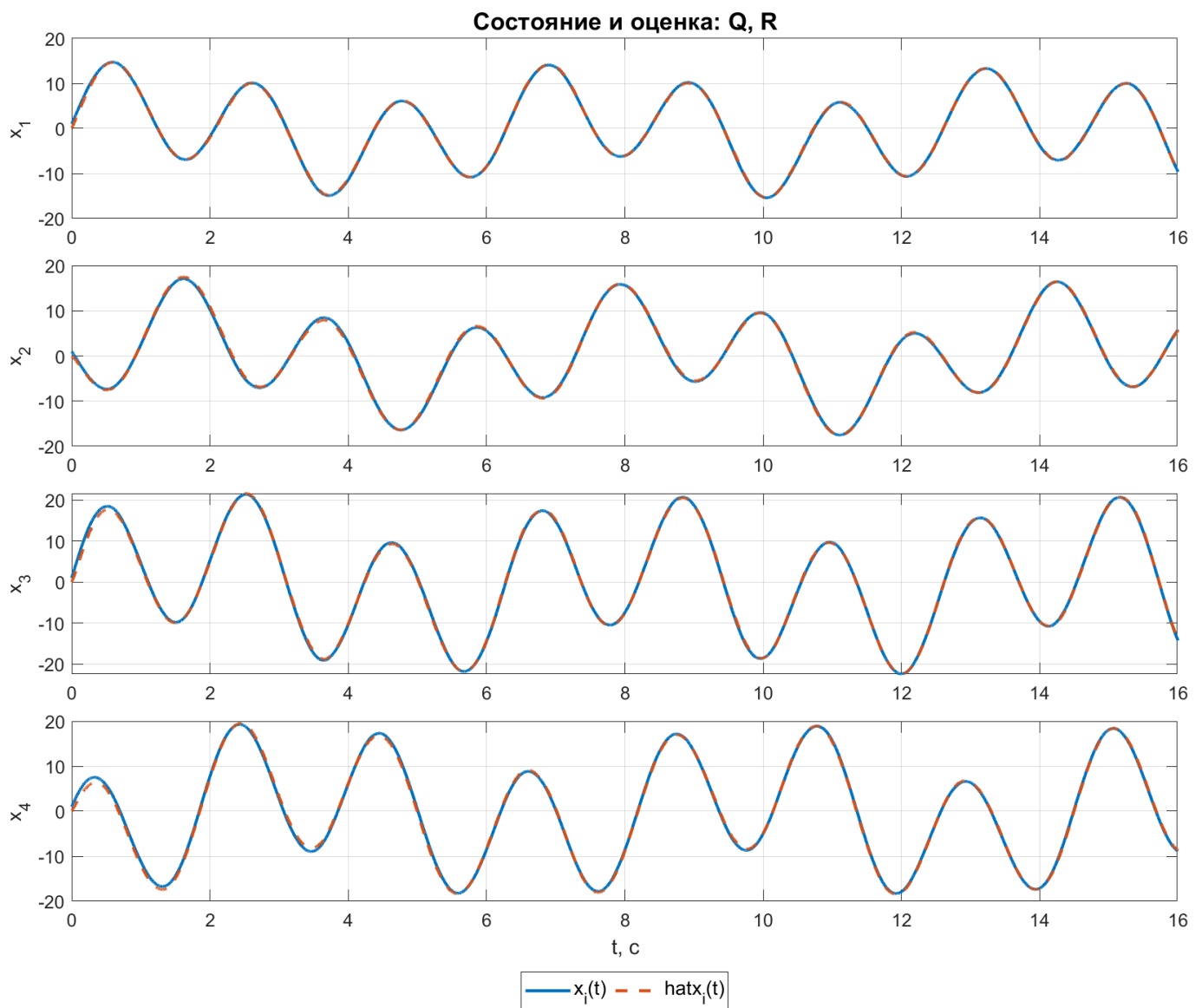


Рисунок 9 — Сравнение $x(t)$ и $\hat{x}(t)$ для пары (Q, R)

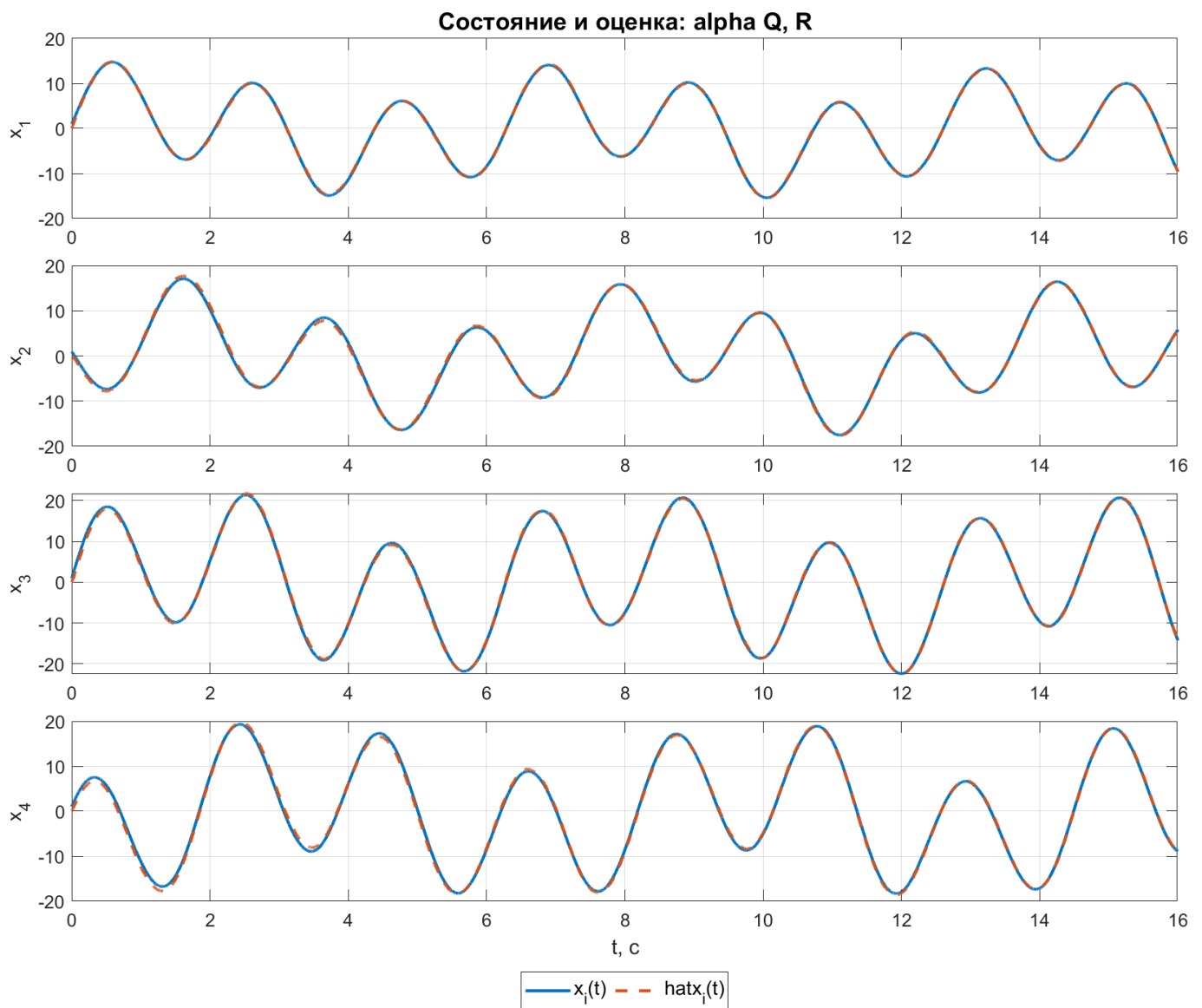


Рисунок 10 — Сравнение $x(t)$ и $\hat{x}(t)$ для пары $(\alpha Q, R)$

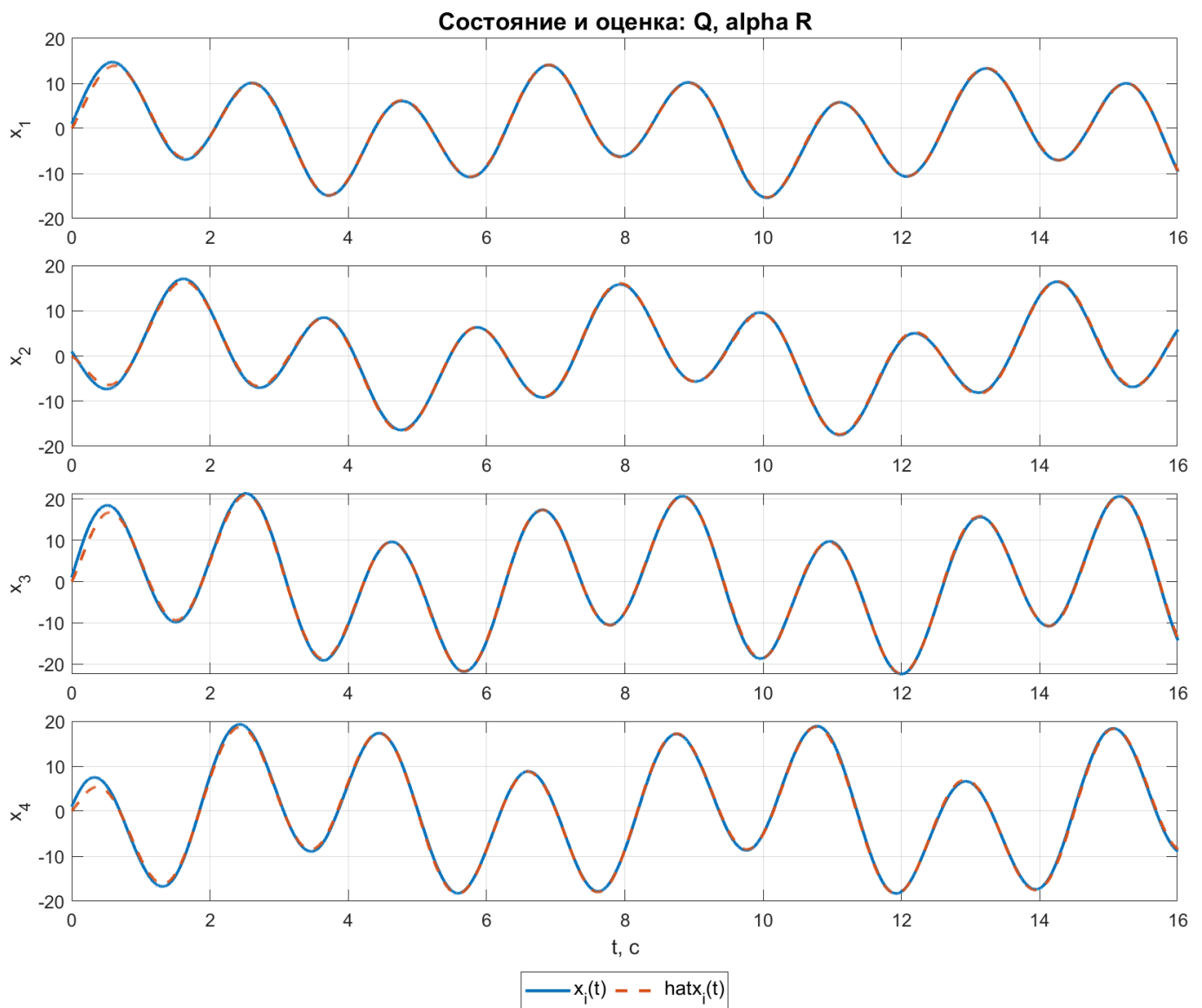


Рисунок 11 — Сравнение $x(t)$ и $\hat{x}(t)$ для пары $(Q, \alpha R)$

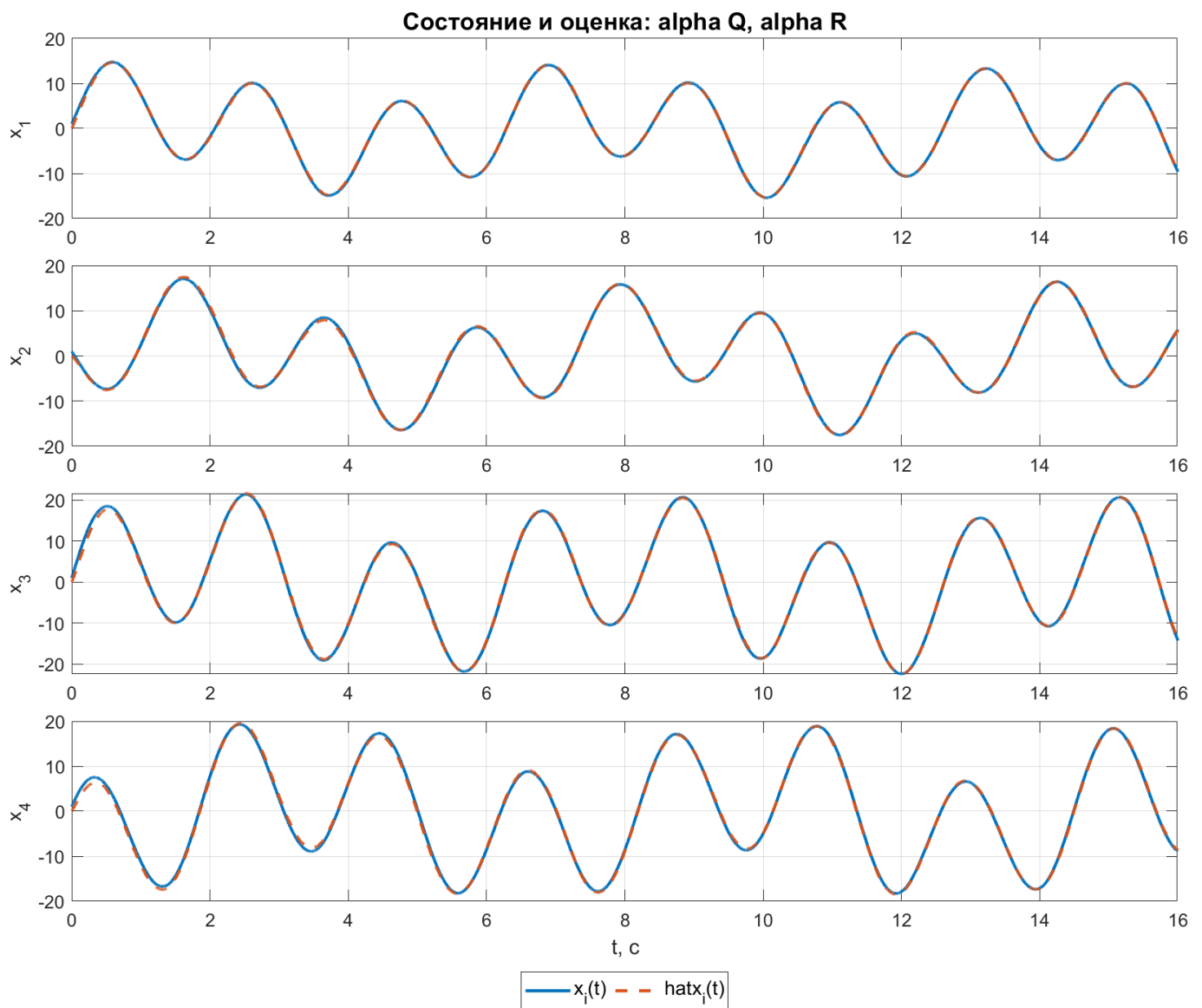


Рисунок 12 — Сравнение $x(t)$ и $\hat{x}(t)$ для пары $(\alpha Q, \alpha R)$

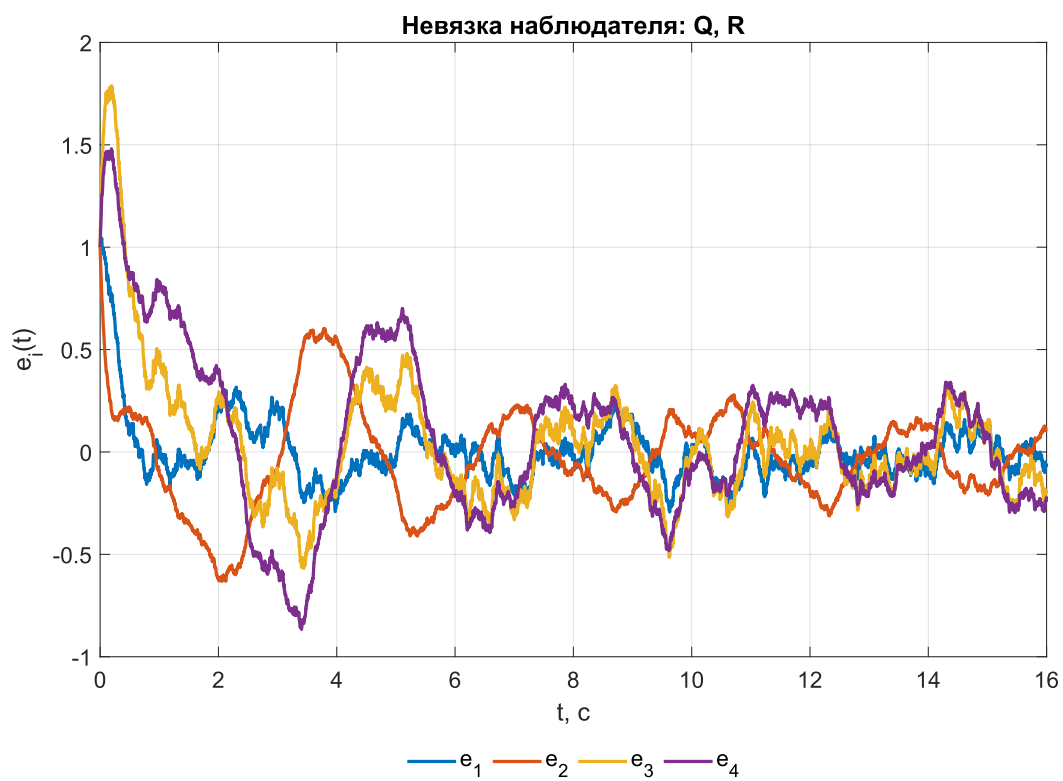


Рисунок 13 — Невязка наблюдателя для пары (Q, R)

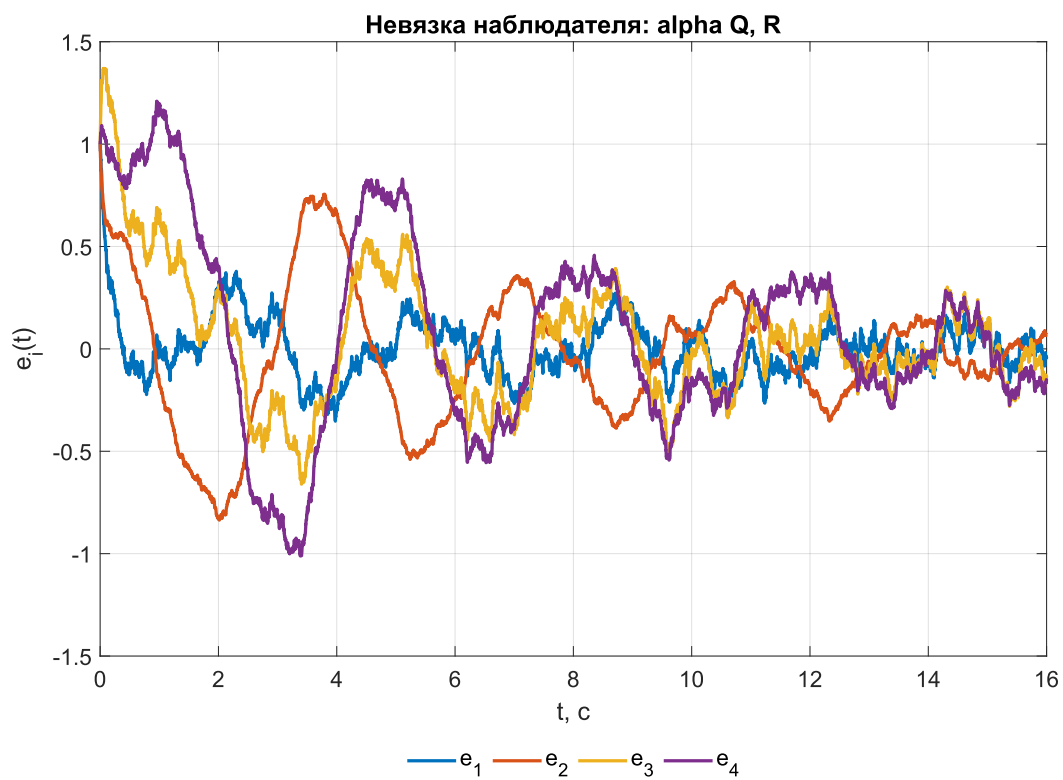


Рисунок 14 — Невязка наблюдателя для пары $(\alpha Q, R)$

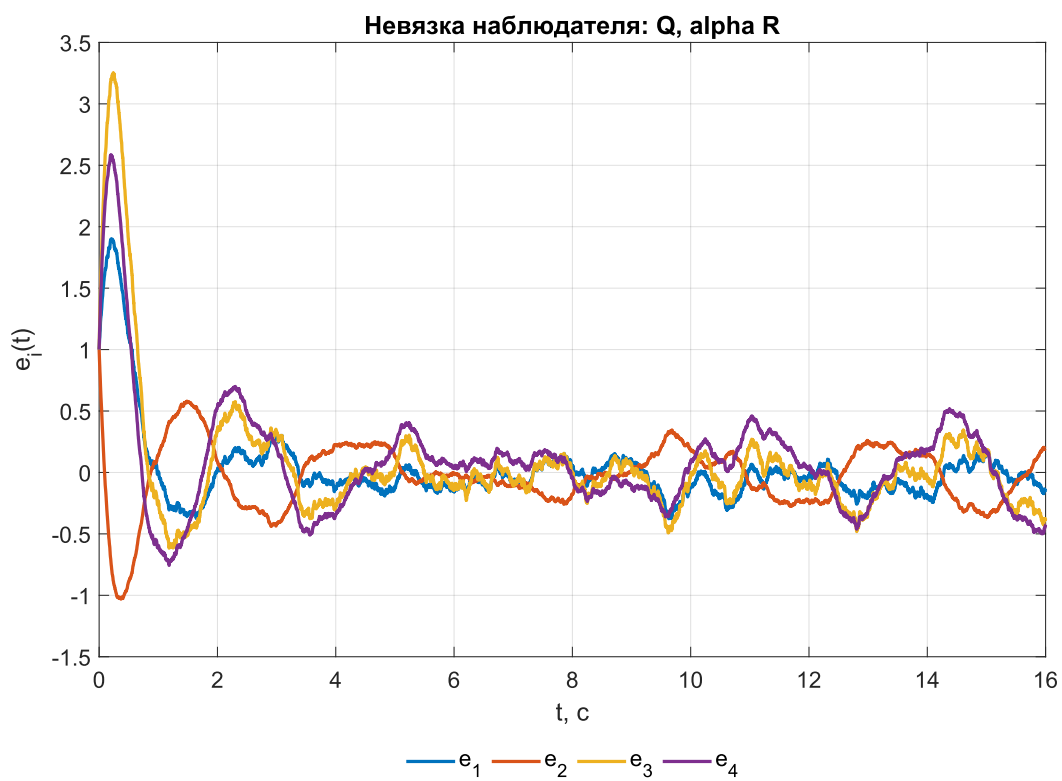


Рисунок 15 — Невязка наблюдателя для пары $(Q, \alpha R)$

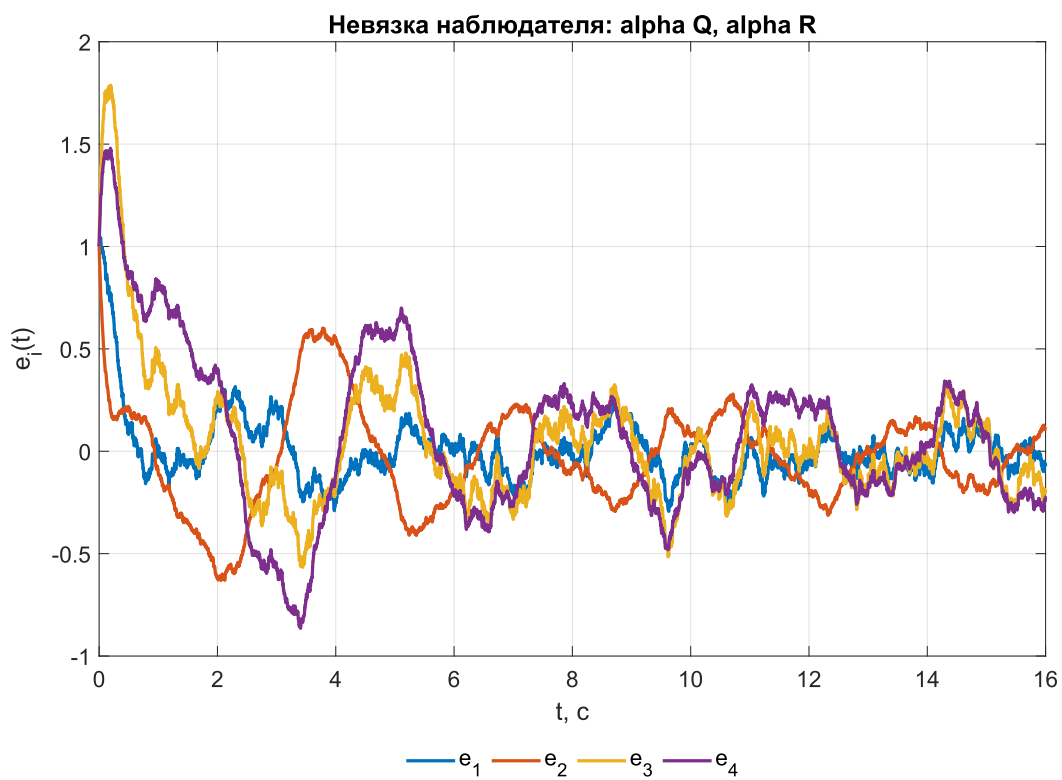


Рисунок 16 — Невязка наблюдателя для пары $(\alpha Q, \alpha R)$

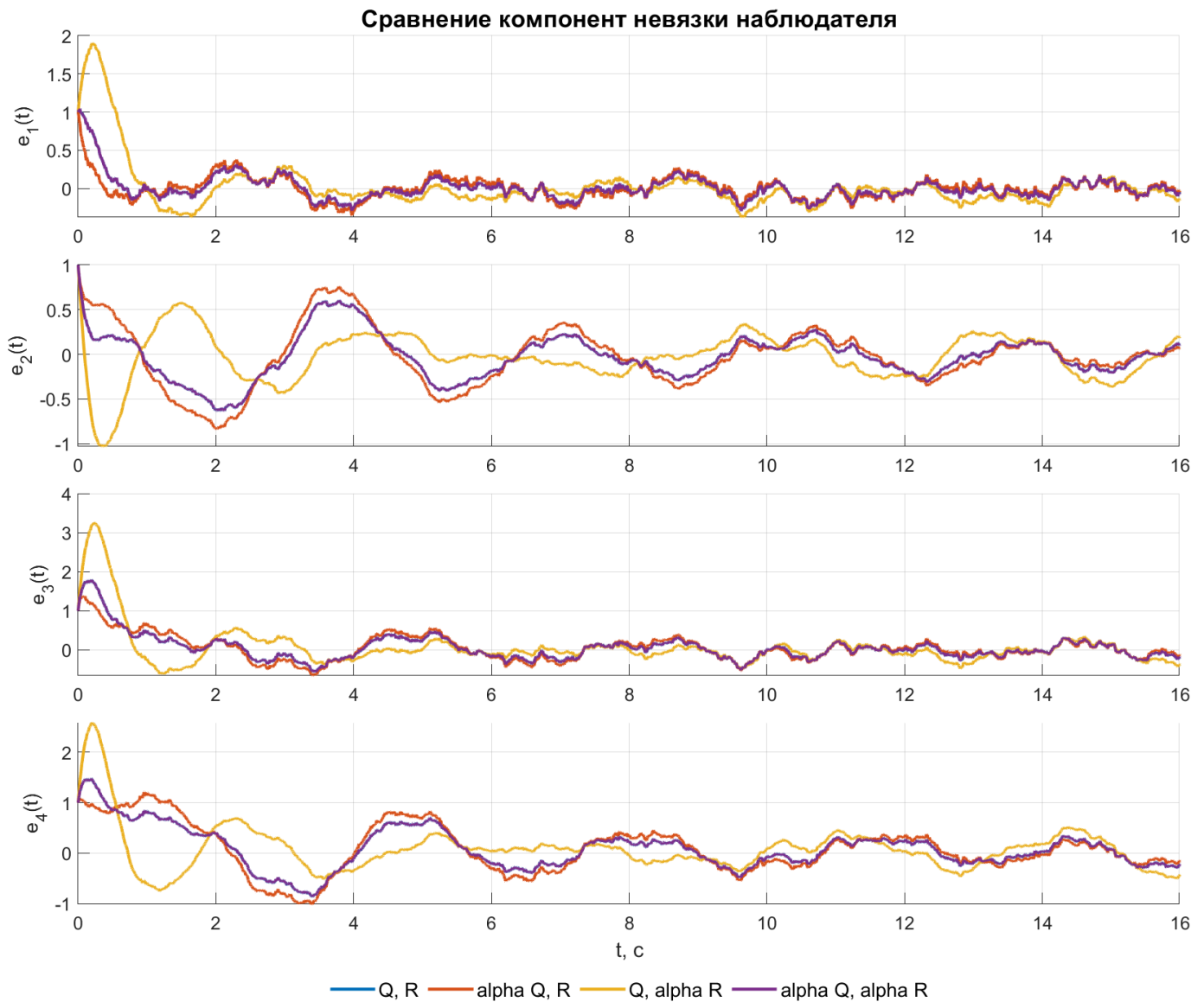


Рисунок 17 — Сравнение компонент невязки $e(t)$

Вывод

Система полностью наблюдаема, поэтому обнаруживаемость выполняется. Увеличение Q при неизменном R приводит к более активной коррекции и уменьшает итоговую невязку для выбранной реализации случайных сигналов. Увеличение R при неизменном Q уменьшает доверие к измерению, коррекция становится слабее, а невязка возрастает. Одновременное масштабирование Q и R не изменяет матрицу L , поэтому динамика ошибки практически совпадает со случаем (Q, R) .

Задание 3

Рассматривается система

$$\begin{cases} \dot{x} = Ax + Bu + f \\ y = Cx + Du + \xi \end{cases} \quad x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad \hat{x}(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

где

$$A = \begin{bmatrix} 6 & 0 & -12 & 6 \\ 0 & 6 & -6 & 12 \\ -12 & -6 & 6 & 0 \\ 6 & 12 & 0 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 6 & 0 \\ 12 & 0 \\ 6 & 0 \\ 4 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} -6 & 6 & 6 & 6 \\ 3 & 0 & 0 & 3 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}.$$

Так как вариант четный, в качестве внешнего возмущения и помехи измерения используются детерминированные гармонические сигналы:

$$f(t) = \begin{bmatrix} \sin 2t \\ \cos 2t \\ 0.5 \sin 3t \\ 0.5 \cos 3t \end{bmatrix}, \quad \xi(t) = \begin{bmatrix} 0.2 \sin 4t \\ 0.2 \cos 4t \end{bmatrix}.$$

Регулятор и наблюдатель объединяются по принципу разделения. Управление формируется по оценке состояния

$$u = K\hat{x},$$

а наблюдатель для системы с прямым прохождением управления в выходе имеет вид

$$\dot{\hat{x}} = A\hat{x} + Bu + L(C\hat{x} + Du - y).$$

Для невязки $e = x - \hat{x}$ получаем

$$C\hat{x} + Du - y = C\hat{x} + Du - Cx - Du - \xi = -Ce - \xi,$$

поэтому

$$\dot{e} = \dot{x} - \dot{\hat{x}} = (A + LC)e + f + L\xi.$$

Следовательно, матрица K определяет собственное движение замкнутой системы по состоянию, а матрица L определяет собственное движение ошибки наблюдателя. Наличие матрицы D не изменяет уравнение ошибки, потому что слагаемое Du известно наблюдателю и сокращается в невязке выхода.

Проверка стабилизируемости и обнаруживаемости

Матрица управляемости равна

$$U = [B \ AB \ A^2B \ A^3B], \quad \text{rank}(U) = 4.$$

Следовательно, система полностью управляема, поэтому она стабилизируема. Собственные числа матрицы A :

$$\sigma(A) = \{-12, 0, 12, 24\}.$$

Условие Хаутуса для управляемости выполняется для всех собственных чисел:

$$\text{rank} [-12I - A \ B] = 4, \quad \text{rank} [0I - A \ B] = 4,$$

$$\text{rank} [12I - A \ B] = 4, \quad \text{rank} [24I - A \ B] = 4.$$

Матрица наблюдаемости имеет ранг

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \\ CA^3 \end{bmatrix}, \quad \text{rank}(O) = 3.$$

Полная наблюдаемость не выполняется, поэтому проверяется обнаруживаемость. Условие Хаутуса по выходу дает

$$\text{rank} \begin{bmatrix} -12I - A \\ C \end{bmatrix} = 3, \quad \text{rank} \begin{bmatrix} 0I - A \\ C \end{bmatrix} = 4,$$

$$\text{rank} \begin{bmatrix} 12I - A \\ C \end{bmatrix} = 4, \quad \text{rank} \begin{bmatrix} 24I - A \\ C \end{bmatrix} = 4.$$

Единственная ненаблюдаемая мода соответствует $\lambda = -12$ и является устойчивой. Следовательно, система обнаруживаема, и матрица L может быть выбрана так, чтобы матрица $A + LC$ была гурвицевой.

Синтез регулятора и наблюдателя

Для регулятора выбраны веса

$$Q_K = I_4, \quad R_K = I_2.$$

Матрица K находится из уравнения Риккати

$$A^T P_K + P_K A + Q_K - P_K B R_K^{-1} B^T P_K = 0, \quad K = -R_K^{-1} B^T P_K.$$

Получено

$$P_K = \begin{bmatrix} 9.4186845991 & 0.1933863068 & -7.8934777660 & 1.6248431399 \\ 0.1933863068 & 0.6160108878 & -0.7459487270 & 0.0738651342 \\ -7.8934777660 & -0.7459487270 & 7.3857041386 & -1.2433056878 \\ 1.6248431399 & 0.0738651342 & -1.2433056878 & 0.5491525863 \end{bmatrix},$$

$$K = \begin{bmatrix} -17.9712492402 & -4.3722166688 & 16.9712492402 & -5.3722166688 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Вторая строка K равна нулю, потому что второй столбец матрицы B нулевой и второе управляющее воздействие не влияет на уравнение состояния. Спектр замкнутой системы по регулятору:

$$\sigma(A + BK) = \{-25.8809371205, -16.7454676932, -12, -1.3290618871\}.$$

Все собственные числа лежат в левой полуплоскости, поэтому регулятор стабилизирует объект.

Для наблюдателя выбраны веса

$$Q_L = I_4, \quad R_L = I_2.$$

Матрица L находится из уравнения Риккати

$$AP_L + P_L A^T + Q_L - P_L C^T R_L^{-1} C P_L = 0, \quad L = -P_L C^T R_L^{-1}.$$

Получено

$$P_L = \begin{bmatrix} 1.4819948774 & 1.5480693123 & -1.6278282107 & 1.3605693123 \\ 1.5480693123 & 2.2343025915 & -2.1128770264 & 1.6278282107 \\ -1.6278282107 & -2.1128770264 & 2.2343025915 & -1.5480693123 \\ 1.3605693123 & 1.6278282107 & -1.5480693123 & 1.4819948774 \end{bmatrix},$$

$$L = \begin{bmatrix} 1.2071067812 & -8.5276925691 \\ -1.2071067812 & -9.5276925691 \\ -1.2071067812 & 9.5276925691 \\ -1.2071067812 & -8.5276925691 \end{bmatrix}.$$

Спектр динамики ошибки наблюдателя:

$$\sigma(A + LC) = \{-2.9764759894, -24.1896794250, -12, -16.9705627485\}.$$

Все собственные числа матрицы $A + LC$ имеют отрицательные вещественные части, поэтому невязка наблюдателя имеет устойчивую собственную динамику. Замкнутая

LQG-система имеет две независимые группы собственных чисел: $\sigma(A + BK)$ и $\sigma(A + LC)$.

Моделирование

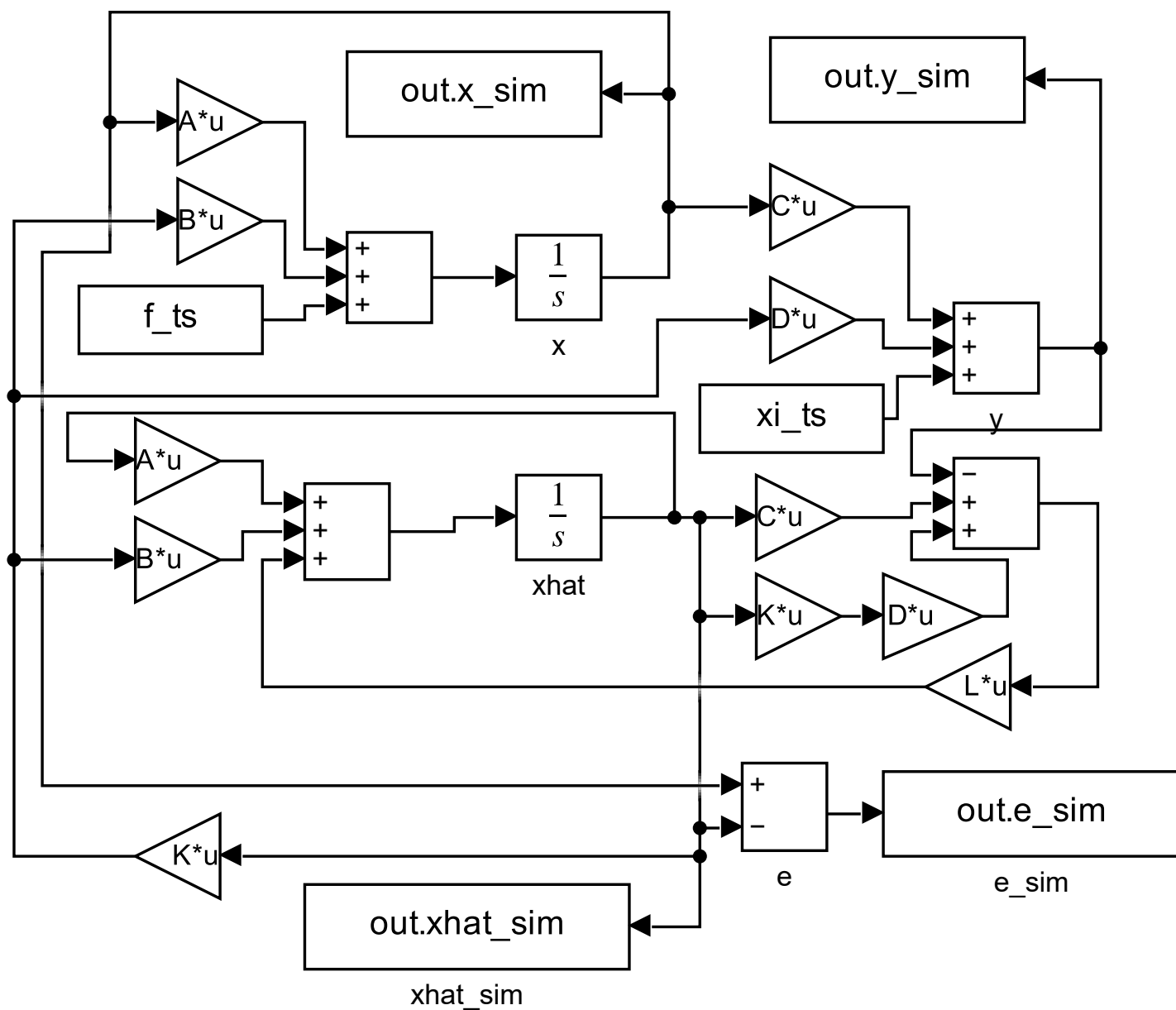


Рисунок 18 — Схема моделирования LQG-системы

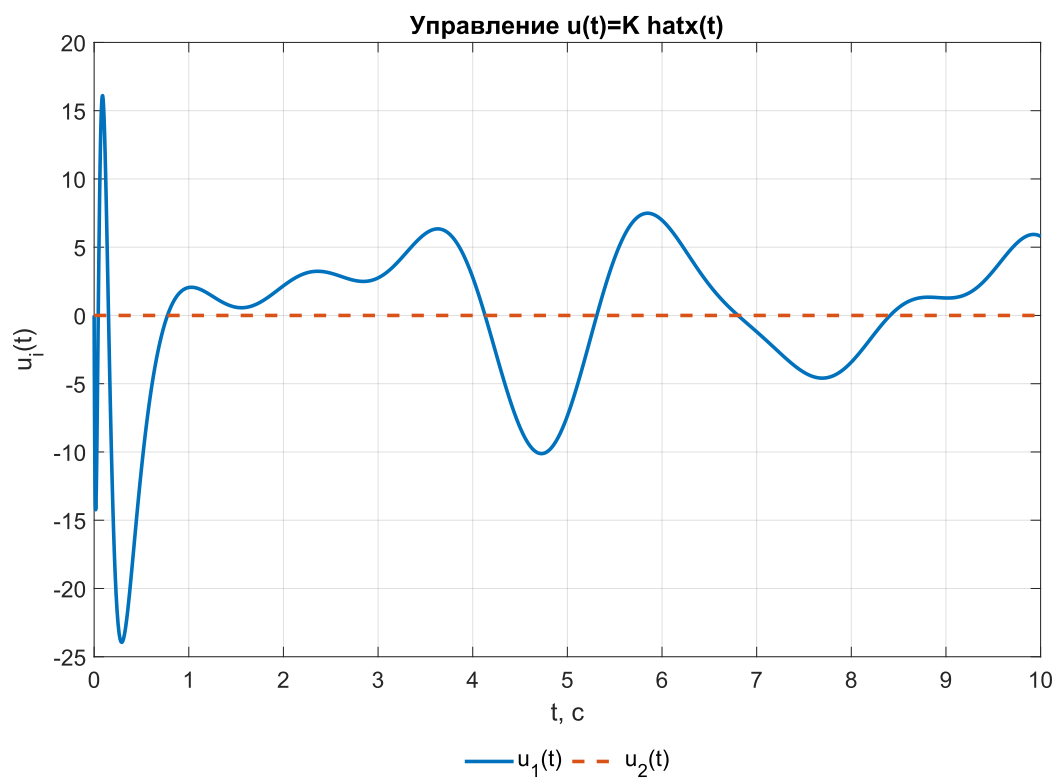


Рисунок 19 — Управление $u(t) = K \hat{x}(t)$

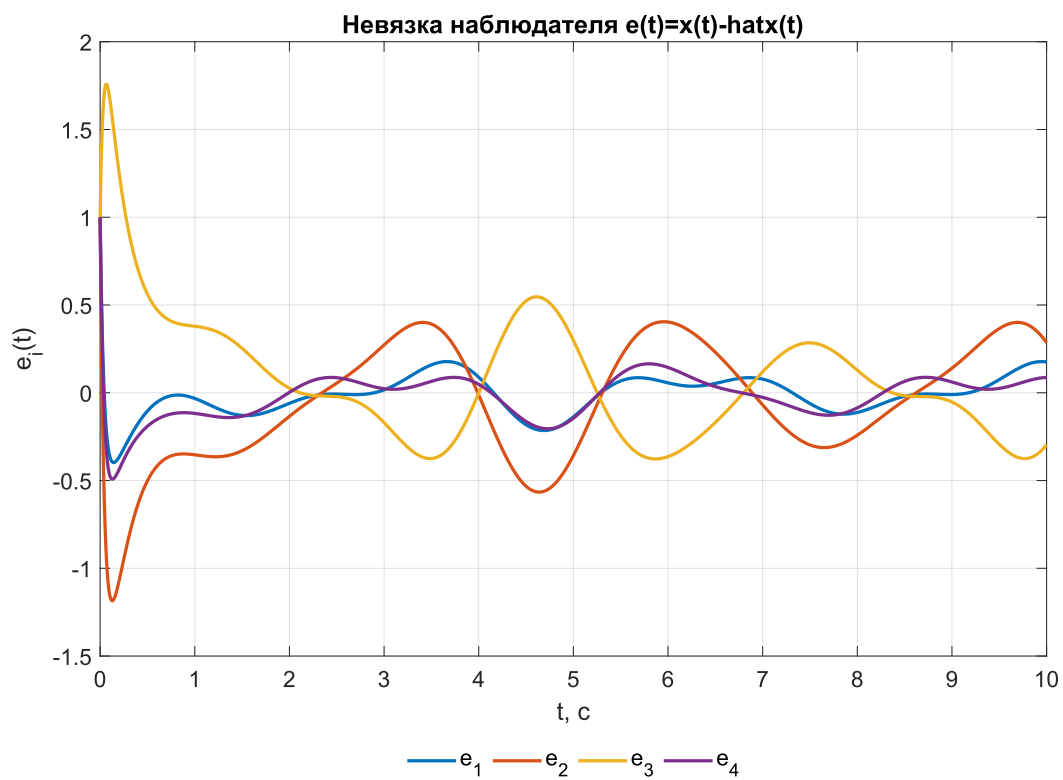


Рисунок 20 — Невязка $e(t) = x(t) - \hat{x}(t)$

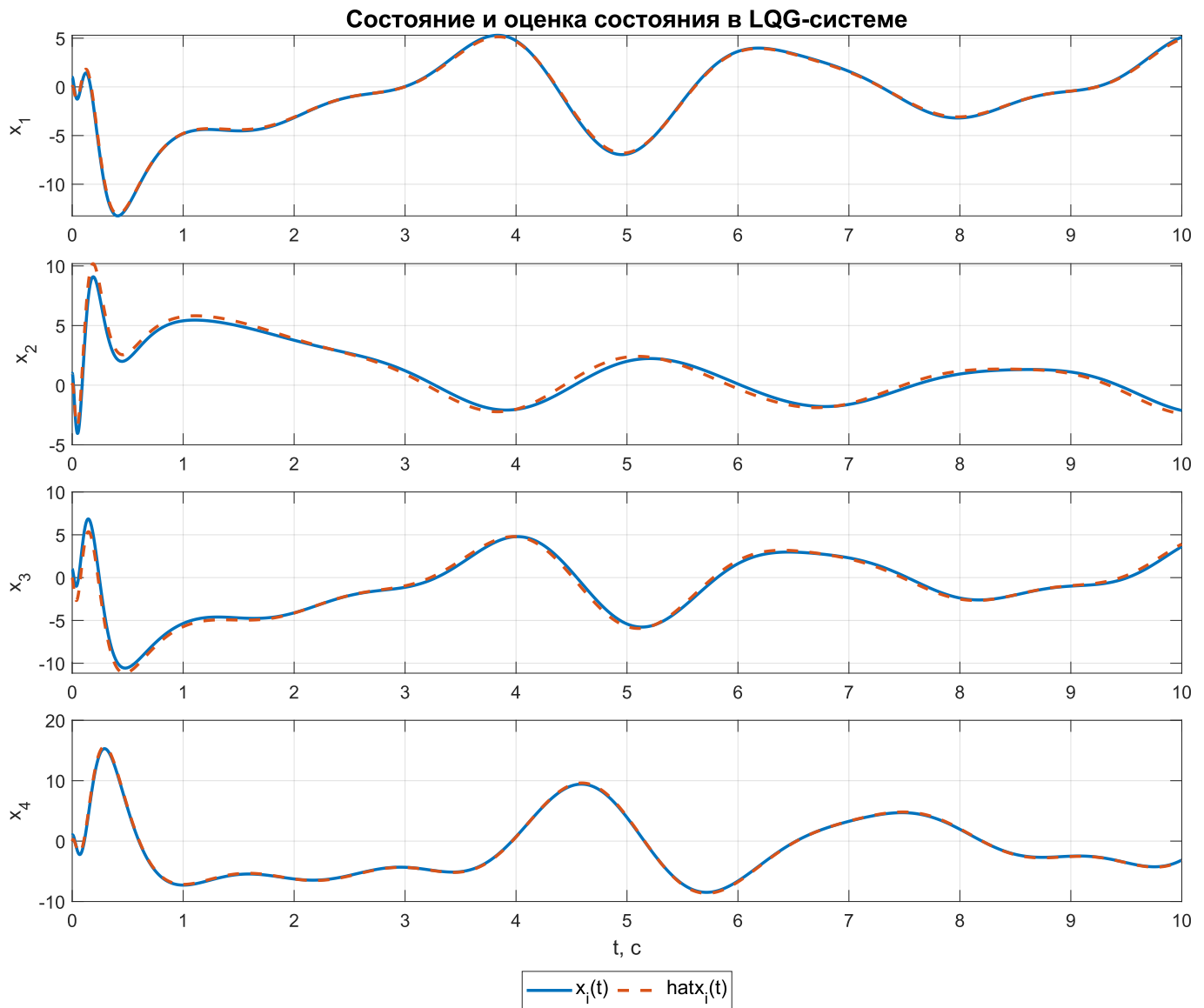


Рисунок 21 — Сравнение $x(t)$ и $\hat{x}(t)$

По результатам моделирования на интервале $[0, 10]$ ошибка наблюдателя не стремится точно к нулю из-за действия внешнего гармонического возмущения $f(t)$ и помехи измерения $\xi(t)$, однако собственная динамика ошибки устойчива, поэтому компоненты $e_{i(t)}$ остаются ограниченными. Сформированное управление также остается ограниченным; для данной реализации сигналов получено $\max_t \|u(t)\| = 23.9518942029$.

Вывод

Для заданной системы выполнены условия существования LQG-регулятора: пара (A, B) управляема, а пара (C, A) обнаруживаема. Матрица K , найденная из уравнения Риккати для регулятора, стабилизирует объект; матрица L , найденная из уравнения Риккати для наблюдателя, делает динамику ошибки устойчивой. Компьютерное моделирование подтверждает, что при гармонических возмущениях состояние, оценка состояния,

управление и невязка остаются ограниченными, а регулятор и наблюдатель совместно формируют устойчивую замкнутую LQG-систему.

Общий вывод

В лабораторной работе были исследованы LQR, LQE и LQG-синтезы, основанные на решении матричных уравнений Риккати. Для регулятора проверялась стабилизируемость пары (A, B) , для наблюдателя – обнаруживаемость пары (C, A) . В первом задании получены регуляторы $u = Kx$, минимизирующие квадратичный функционал качества; во втором задании синтезирован наблюдатель, уменьшающий влияние возмущения и шума на ошибку оценивания; в третьем задании регулятор и наблюдатель объединены по принципу разделения в LQG-систему с управлением $u = K\hat{x}$. Моделирование подтвердило устойчивость полученных замкнутых систем и ограниченность сигналов состояния, оценки, управления и невязки.

Приложение

MATLAB-скрипт для задания 1

```
1  clear; clc; close all;
2
3  scriptDir = fileparts(mfilename('fullpath'));
4  cd(scriptDir);
5
6  modelName = 'first_model';
7  modelPath = fullfile(scriptDir, 'first_task', [modelName '.slx']);
8  outDir = fullfile(scriptDir, 'first_task', 'outputs');
9  if ~exist(outDir, 'dir')
10     mkdir(outDir);
11 end
12
13 A = [5 6 3; -6 -7 -6; 6 6 5];
14 B = [1; 0; 0];
15 x0 = [1; 1; 1];
16
17 Q0 = eye(3);
18 R0 = 1;
19 alpha = 5;
20 nu = 1;
21 tEnd = 12;
22
23 cases = {
24     'QR', Q0, R0, 'Q, R';
25     'aQR', alpha * Q0, R0, 'alpha Q, R';
26     'QaR', Q0, alpha * R0, 'Q, alpha R';
27     'aQaR', alpha * Q0, alpha * R0, 'alpha Q, alpha R';
28 };
29
30 U = [B A*B A^2*B];
31 eigA = eig(A);
32 hautusRanks = zeros(numel(eigA), 1);
33 for i = 1:numel(eigA)
34     hautusRanks(i) = rank([eigA(i) * eye(3) - A, B]);
```

Matlab

```

35 end
36
37 results = struct([]);
38 for i = 1:size(cases, 1)
39     Q = cases{i, 2};
40     R = cases{i, 3};
41     [P, ~, ~] = care(A, B, Q, R);
42     K = -(R \ (B' * P));
43     Acl = A + B * K;
44     Jmin = x0' * P * x0;
45
46     results(i).id = cases{i, 1};
47     results(i).label = cases{i, 4};
48     results(i).Q = Q;
49     results(i).R = R;
50     results(i).P = P;
51     results(i).K = K;
52     results(i).Acl = Acl;
53     results(i).eigAcl = eig(Acl);
54     results(i).Jmin = Jmin;
55 end
56
57 if ~(exist(modelPath, 'file') == 2)
58     error('Не найдена модель %s', modelPath);
59 end
60
61 load_system(modelPath);
62 set_param(modelName, 'StopTime', num2str(tEnd));
63 export_model_scheme(modelName, fullfile(outDir,
64     'first_task_scheme.svg'));
64
65 for i = 1:numel(results)
66     Q = results(i).Q;
67     R = results(i).R;
68     K = results(i).K;
69
70     assignin('base', 'A', A);
71     assignin('base', 'B', B);

```

```

72     assignin('base', 'K', K);
73     assignin('base', 'x0', x0);
74     assignin('base', 'tEnd', tEnd);
75
76     simOut = sim(modelName, 'SrcWorkspace', 'current', 'StopTime',
77         num2str(tEnd));
78     data = collect_data(simOut);
79     data.Jexp = cumulative_cost(data.t, data.x, data.u, Q, R);
80
81     results(i).t = data.t;
82     results(i).x = data.x;
83     results(i).u = data.u;
84     results(i).Jexp = data.Jexp;
85
86     save_case_plots(outDir, data, results(i).id, results(i).label,
87         results(i).Jmin);
88
89     end
90
91     save_common_plots(outDir, results);
92     write_summary(fullfile(outDir, 'task1_summary.txt'), A, B, x0, Q0, R0,
93         alpha, nu, U, eigA, hautusRanks, results);
94     save(fullfile(outDir, 'task1_results.mat'), 'A', 'B', 'x0', 'Q0', 'R0',
95         'alpha', 'nu', 'U', 'eigA', 'hautusRanks', 'results');
96
97     save_system(modelName);
98     bdclose(modelName);
99
100    function data = collect_data(simOut)
101        data.t = simOut.tout(:);
102        data.x = read_workspace_signal(simOut, 'x_sim');
103        data.u = read_workspace_signal(simOut, 'u_sim');
104        if size(data.x, 1) ~= numel(data.t)
105            data.x = data.x.';
106        end
107        if size(data.u, 1) ~= numel(data.t)
108            data.u = data.u.';
109        end
110    end
111 end

```



```

106
107 function y = read_workspace_signal(simOut, name)
108     raw = simOut.get(name);
109     if isa(raw, 'timeseries')
110         y = raw.Data;
111     elseif isstruct(raw) && isfield(raw, 'signals')
112         y = raw.signals.values;
113     else
114         y = raw;
115     end
116     y = squeeze(y);
117 end
118
119 function J = cumulative_cost(t, x, u, Q, R)
120     n = numel(t);
121     integrand = zeros(n, 1);
122     for k = 1:n
123         xk = x(k, :).';
124         uk = u(k, :).';
125         integrand(k) = xk' * Q * xk + uk' * R * uk;
126     end
127     J = cumtrapz(t, integrand);
128 end
129
130 function save_case_plots(outDir, data, id, labelText, Jmin)
131     fig = figure('Visible', 'off', 'Color', 'w');
132     set(fig, 'Position', [100, 100, 1000, 720]);
133     plot(data.t, data.x, 'LineWidth', 2.2);
134     grid on;
135     xlabel('t, c', 'FontSize', 16);
136     ylabel('x_i(t)', 'FontSize', 16);
137     title(['Вектор состояния: ' labelText], 'FontSize', 18);
138     lgd = legend('x_1', 'x_2', 'x_3', 'Location', 'southoutside',
139         'Orientation', 'horizontal', 'FontSize', 14);
140     place_legend_below(lgd);
141     set(gca, 'FontSize', 14);
142     save_svg(fig, fullfile(outDir, ['x_' id '.svg']));
143     close(fig);

```

```

143
144     fig = figure('Visible', 'off', 'Color', 'w');
145     set(fig, 'Position', [100, 100, 1000, 720]);
146     plot(data.t, data.u, 'LineWidth', 2.2);
147     grid on;
148     xlabel('t, c', 'FontSize', 16);
149     ylabel('u(t)', 'FontSize', 16);
150     title(['Управление: ' labelText], 'FontSize', 18);
151     lgd = legend(labelText, 'Location', 'southoutside', 'Orientation',
152                 'horizontal', 'FontSize', 14);
152     place_legend_below(lgd);
153     set(gca, 'FontSize', 14);
154     save_svg(fig, fullfile(outDir, ['u_' id '.svg']));
155     close(fig);
156
157     fig = figure('Visible', 'off', 'Color', 'w');
158     set(fig, 'Position', [100, 100, 1000, 720]);
159     plot(data.t, data.Jexp, 'LineWidth', 2.2);
160     yline(Jmin, '--', 'LineWidth', 2.0);
161     grid on;
162     xlabel('t, c', 'FontSize', 16);
163     ylabel('J_{exp}(t)', 'FontSize', 16);
164     title(['Экспериментальный функционал: ' labelText], 'FontSize', 18);
165     lgd = legend('J_{exp}(t)', 'J_{min}', 'Location', 'southoutside',
166                 'Orientation', 'horizontal', 'FontSize', 14);
166     place_legend_below(lgd);
167     set(gca, 'FontSize', 14);
168     save_svg(fig, fullfile(outDir, ['J_' id '.svg']));
169     close(fig);
170 end
171
172 function save_common_plots(outDir, results)
173     fig = figure('Visible', 'off', 'Color', 'w');
174     set(fig, 'Position', [100, 100, 1100, 760]);
175     hold on;
176     for i = 1:numel(results)
177         plot(results(i).t, results(i).u, 'LineWidth', 2.2);
178     end

```

```

179     grid on;
180     xlabel('t, c', 'FontSize', 16);
181     ylabel('u(t)', 'FontSize', 16);
182     title('Сравнение управлений', 'FontSize', 18);
183     lgd = legend({results.label}, 'Location', 'southoutside',
184                 'Orientation', 'horizontal', 'FontSize', 14);
184     place_legend_below(lgd);
185     set(gca, 'FontSize', 14);
186     save_svg(fig, fullfile(outDir, 'u_all.svg'));
187     close(fig);
188
189     fig = figure('Visible', 'off', 'Color', 'w');
190     set(fig, 'Position', [100, 100, 1100, 820]);
191     hold on;
192     for i = 1:numel(results)
193         plot(results(i).t, results(i).Jexp, 'LineWidth', 2.2);
194         yline(results(i).Jmin, '--', 'LineWidth', 1.4);
195     end
196     grid on;
197     xlabel('t, c', 'FontSize', 16);
198     ylabel('J_{exp}(t)', 'FontSize', 16);
199     title('Сравнение функционалов качества', 'FontSize', 18);
200     lgd = legend(compose_legend(results), 'Location', 'southoutside',
201                 'FontSize', 12);
201     try
202         lgd.NumColumns = 2;
203     catch
204     end
205     place_legend_below(lgd);
206     set(gca, 'FontSize', 14);
207     save_svg(fig, fullfile(outDir, 'J_all.svg'));
208     close(fig);
209 end
210
211 function labels = compose_legend(results)
212     labels = cell(1, 2 * numel(results));
213     for i = 1:numel(results)
214         labels{2*i - 1} = ['J_{exp}: ' results(i).label];

```

```

215         labels{2*i} = ['J_{min}': ' results(i).label'];
216     end
217 end
218
219 function write_summary(filePath, A, B, x0, Q0, R0, alpha, nu, U, eigA,
    hautusRanks, results)
220     fid = fopen(filePath, 'w');
221     fprintf(fid, 'A =\n'); write_matrix(fid, A);
222     fprintf(fid, 'B =\n'); write_matrix(fid, B);
223     fprintf(fid, 'x0 =\n'); write_matrix(fid, x0);
224     fprintf(fid, 'Q =\n'); write_matrix(fid, Q0);
225     fprintf(fid, 'R =\n'); write_matrix(fid, R0);
226     fprintf(fid, 'alpha = %.12g\n', alpha);
227     fprintf(fid, 'nu = %.12g\n', nu);
228     fprintf(fid, '\nU = [B AB A^2B] =\n'); write_matrix(fid, U);
229     fprintf(fid, 'rank(U) = %d\n', rank(U));
230     fprintf(fid, 'eig(A) =\n'); write_vector_complex(fid, eigA);
231     fprintf(fid, 'Hautus ranks:\n');
232     for i = 1:numel(eigA)
233         fprintf(fid, 'lambda = %+.12g%+.12gi, rank = %d\n',
            real(eigA(i)), imag(eigA(i)), hautusRanks(i));
234     end
235     for i = 1:numel(results)
236         fprintf(fid, '\nCase %s\n', results(i).label);
237         fprintf(fid, 'Q =\n'); write_matrix(fid, results(i).Q);
238         fprintf(fid, 'R =\n'); write_matrix(fid, results(i).R);
239         fprintf(fid, 'P =\n'); write_matrix(fid, results(i).P);
240         fprintf(fid, 'K =\n'); write_matrix(fid, results(i).K);
241         fprintf(fid, 'eig(A+B*K) =\n'); write_vector_complex(fid,
            results(i).eigAcl);
242         fprintf(fid, 'Jmin = %.12g\n', results(i).Jmin);
243         if isfield(results(i), 'Jexp')
244             fprintf(fid, 'Jexp(tEnd) = %.12g\n', results(i).Jexp(end));
245         end
246     end
247     fclose(fid);
248 end
249

```

```

250 function write_matrix(fid, M)
251     for i = 1:size(M, 1)
252         fprintf(fid, ' ');
253         fprintf(fid, '%.12g ', M(i, :));
254         fprintf(fid, '\n');
255     end
256 end
257
258 function write_vector_complex(fid, v)
259     for i = 1:numel(v)
260         fprintf(fid, '%+.12g%+.12gi\n', real(v(i)), imag(v(i)));
261     end
262 end
263
264 function export_model_scheme(modelName, outPath)
265     open_system(modelName);
266     try
267         print(['-s' modelName], '-dsvg', outPath);
268     catch
269         saveas(get_param(modelName, 'Handle'), outPath);
270     end
271 end
272
273 function save_svg(fig, outPath)
274     set(fig, 'PaperPositionMode', 'auto');
275     print(fig, outPath, '-dsvg');
276 end
277
278 function place_legend_below(lgd)
279     try
280         lgd.Box = 'off';
281         lgd.Units = 'normalized';
282     catch
283     end
284 end

```

MATLAB-скрипт для задания 2

```
1  clear; clc; close all;
2
3  scriptDir = fileparts(mfilename('fullpath'));
4  cd(scriptDir);
5
6  taskDir = fullfile(scriptDir, 'second_task');
7  modelFileName = 'second_model_original';
8  modelPath = fullfile(taskDir, [modelFileName '.slx']);
9  [~, modelName] = fileparts(modelPath);
10 outDir = fullfile(taskDir, 'outputs');
11 if ~exist(outDir, 'dir')
12     mkdir(outDir);
13 end
14
15 A = [35 10 -28 17;
16      -22 -7 18 -12;
17      56 17 -45 27;
18      34 12 -28 17];
19 C = [7 2 -5 3];
20 x0 = [1; 1; 1; 1];
21 xhat0 = zeros(4, 1);
22
23 Q0 = eye(4);
24 R0 = 1;
25 alpha = 5;
26 nu = 1;
27 tEnd = 16;
28 dt = 0.002;
29 t = (0:dt:tEnd).';
30
31 rng(10);
32 f = randn(numel(t), 4);
33 xi = randn(numel(t), 1);
34 f_ts = timeseries(f, t);
35 xi_ts = timeseries(xi, t);
36
```

 Matlab

```

37  cases = {
38      'QR', Q0, R0, 'Q, R';
39      'aQR', alpha * Q0, R0, 'alpha Q, R';
40      'QaR', Q0, alpha * R0, 'Q, alpha R';
41      'aQaR', alpha * Q0, alpha * R0, 'alpha Q, alpha R';
42  };
43
44  0 = obsv(A, C);
45  eigA = eig(A);
46  hautusRanks = zeros(numel(eigA), 1);
47  for i = 1:numel(eigA)
48      hautusRanks(i) = rank([eigA(i) * eye(4) - A; C]);
49  end
50
51  results = struct([]);
52  for i = 1:size(cases, 1)
53      Q = cases{i, 2};
54      R = cases{i, 3};
55      P = care(A.', C.', Q, R);
56      L = -(P * C.') / R;
57      Aerr = A + L * C;
58
59      results(i).id = cases{i, 1};
60      results(i).label = cases{i, 4};
61      results(i).Q = Q;
62      results(i).R = R;
63      results(i).P = P;
64      results(i).L = L;
65      results(i).Aerr = Aerr;
66      results(i).eigAerr = eig(Aerr);
67  end
68
69  if bdIsLoaded(modelName)
70      close_system(modelName, 0);
71  end
72  load_system(modelPath);
73  set_param(modelName, 'StopTime', 'tEnd');

```

```

74  set_param(modelName, 'SolverType', 'Variable-step', 'Solver', 'ode45',
    'RelTol', '1e-7', 'AbsTol', '1e-9');
75  check_second_model(modelName);
76  export_model_scheme(modelName, fullfile(outDir,
    'second_task_scheme.svg'));
77
78  assignin('base', 'A', A);
79  assignin('base', 'C', C);
80  assignin('base', 'x0', x0);
81  assignin('base', 'xhat0', xhat0);
82  assignin('base', 'f_ts', f_ts);
83  assignin('base', 'xi_ts', xi_ts);
84  assignin('base', 'tEnd', tEnd);
85
86  for i = 1:numel(results)
87      L = results(i).L;
88      assignin('base', 'L', L);
89
90      simOut = sim(modelName, 'SrcWorkspace', 'base', 'StopTime',
        num2str(tEnd));
91      data = collect_data(simOut);
92      results(i).t = data.t;
93      results(i).x = data.x;
94      results(i).xhat = data.xhat;
95      results(i).e = data.e;
96      results(i).y = data.y;
97      results(i).eNormMax = max(vecnorm(data.e, 2, 2));
98      results(i).eNormEnd = norm(data.e(end, :));
99
100     save_case_plots(outDir, data, results(i).id, results(i).label);
101 end
102
103 save_common_plots(outDir, results);
104 write_summary(fullfile(outDir, 'task2_summary.txt'), A, C, x0, xhat0, Q0,
    R0, alpha, nu, 0, eigA, hautusRanks, results);
105 save(fullfile(outDir, 'task2_results.mat'), 'A', 'C', 'x0', 'xhat0',
    'Q0', 'R0', 'alpha', 'nu', '0', 'eigA', 'hautusRanks', 'results');
106

```



```

107 bdclose(modelName);
108
109 function check_second_model(modelName)
110     checks = {
111         'FromWorkspace', 'VariableName', 'f_ts';
112         'FromWorkspace', 'VariableName', 'xi_ts';
113         'Gain', 'Gain', 'A';
114         'Gain', 'Gain', 'C';
115         'Gain', 'Gain', 'L';
116         'Integrator', 'InitialCondition', 'x0';
117         'Integrator', 'InitialCondition', 'xhat0';
118         'ToWorkspace', 'VariableName', 'x_sim';
119         'ToWorkspace', 'VariableName', 'xhat_sim';
120         'ToWorkspace', 'VariableName', 'e_sim';
121         'ToWorkspace', 'VariableName', 'y_sim';
122     };
123     for i = 1:size(checks, 1)
124         blocks = find_system(modelName, 'SearchDepth', 1, 'BlockType',
125             checks{i, 1}, checks{i, 2}, checks{i, 3});
126         if isempty(blocks)
127             error('В модели отсутствует блок типа "%s" с параметром
128                 "%s=%s".', ...
129                 checks{i, 1}, checks{i, 2}, checks{i, 3});
130         end
131     end
132 end
133
134 function data = collect_data(simOut)
135     data.t = simOut.tout(:);
136     data.x = normalize_signal(read_workspace_signal(simOut, 'x_sim'),
137         numel(data.t));
138     data.xhat = normalize_signal(read_workspace_signal(simOut,
139         'xhat_sim'), numel(data.t));
140     data.e = normalize_signal(read_workspace_signal(simOut, 'e_sim'),
141         numel(data.t));
142     data.y = normalize_signal(read_workspace_signal(simOut, 'y_sim'),
143         numel(data.t));
144 end

```

```

139
140 function y = read_workspace_signal(simOut, name)
141     raw = simOut.get(name);
142     if isa(raw, 'timeseries')
143         y = raw.Data;
144     elseif isstruct(raw) && isfield(raw, 'signals')
145         y = raw.signals.values;
146     else
147         y = raw;
148     end
149     y = squeeze(y);
150 end
151
152 function y = normalize_signal(y, n)
153     if isvector(y)
154         y = y(:);
155     end
156     if size(y, 1) ~= n
157         y = y.';
158     end
159 end
160
161 function save_case_plots(outDir, data, id, labelText)
162     fig = figure('Visible', 'off', 'Color', 'w');
163     set(fig, 'Position', [100, 100, 1100, 900]);
164     tiledlayout(4, 1, 'TileSpacing', 'compact', 'Padding', 'compact');
165     for k = 1:4
166         nexttile;
167         plot(data.t, data.x(:, k), 'LineWidth', 2.0); hold on;
168         plot(data.t, data.xhat(:, k), '--', 'LineWidth', 2.0);
169         grid on;
170         ylabel(sprintf('x_%d', k), 'FontSize', 13);
171         set(gca, 'FontSize', 12);
172         if k == 1
173             title(['Состояние и оценка: ' labelText], 'FontSize', 16);
174         end
175         if k == 4

```

```

176         xlabel('t, c', 'FontSize', 14);
177     end
178 end
179     lgd = legend('x_i(t)', 'hat{x}_i(t)', 'Location', 'southoutside',
180         'Orientation', 'horizontal', 'FontSize', 13);
181     lgd.Layout.Tile = 'south';
182     save_svg(fig, fullfile(outDir, ['x_xhat_' id '.svg']));
183     close(fig);
184
185     fig = figure('Visible', 'off', 'Color', 'w');
186     set(fig, 'Position', [100, 100, 1000, 720]);
187     plot(data.t, data.e, 'LineWidth', 2.0);
188     grid on;
189     xlabel('t, c', 'FontSize', 16);
190     ylabel('e_i(t)', 'FontSize', 16);
191     title(['Невязка наблюдателя: ' labelText], 'FontSize', 18);
192     lgd = legend('e_1', 'e_2', 'e_3', 'e_4', 'Location', 'southoutside',
193         'Orientation', 'horizontal', 'FontSize', 14);
194     lgd.Box = 'off';
195     set(gca, 'FontSize', 14);
196     save_svg(fig, fullfile(outDir, ['e_' id '.svg']));
197     close(fig);
198 end
199
200 function save_common_plots(outDir, results)
201     fig = figure('Visible', 'off', 'Color', 'w');
202     set(fig, 'Position', [100, 100, 1100, 900]);
203     tiledlayout(4, 1, 'TileSpacing', 'compact', 'Padding', 'compact');
204     for k = 1:4
205         nexttile;
206         hold on;
207         for i = 1:numel(results)
208             plot(results(i).t, results(i).e(:, k), 'LineWidth', 2.0);
209         end
210         grid on;
211         ylabel(sprintf('e_%d(t)', k), 'FontSize', 13);
212         set(gca, 'FontSize', 12);
213         if k == 1

```

```

212         title('Сравнение компонент невязки наблюдателя', 'FontSize',
213               16);
214     end
215     if k == 4
216         xlabel('t, c', 'FontSize', 14);
217     end
218     lgd = legend({results.label}, 'Location', 'southoutside',
219                 'Orientation', 'horizontal', 'FontSize', 13);
220     lgd.Box = 'off';
221     try
222         lgd.Layout.Tile = 'south';
223     catch
224     end
225     save_svg(fig, fullfile(outDir, 'e_all.svg'));
226     close(fig);
227 end
228 function write_summary(filePath, A, C, x0, xhat0, Q0, R0, alpha, nu, 0,
229 eigA, hautusRanks, results)
230     fid = fopen(filePath, 'w');
231     fprintf(fid, 'A =\n'); write_matrix(fid, A);
232     fprintf(fid, 'C =\n'); write_matrix(fid, C);
233     fprintf(fid, 'x0 =\n'); write_matrix(fid, x0);
234     fprintf(fid, 'xhat0 =\n'); write_matrix(fid, xhat0);
235     fprintf(fid, 'Q* =\n'); write_matrix(fid, Q0);
236     fprintf(fid, 'R* =\n'); write_matrix(fid, R0);
237     fprintf(fid, 'alpha = %.12g\n', alpha);
238     fprintf(fid, 'nu = %.12g\n', nu);
239     fprintf(fid, '\n0 = [C; CA; CA^2; CA^3] =\n'); write_matrix(fid, 0);
240     fprintf(fid, 'rank(0) = %d\n', rank(0));
241     fprintf(fid, 'eig(A) =\n'); write_vector_complex(fid, eigA);
242     fprintf(fid, 'Hautus observability ranks:\n');
243     for i = 1:numel(eigA)
244         fprintf(fid, 'lambda = %+.12g%+.12gi, rank = %d\n',
245               real(eigA(i)), imag(eigA(i)), hautusRanks(i));
246     end
247     for i = 1:numel(results)

```

```

246     fprintf(fid, '\nCase %s\n', results(i).label);
247     fprintf(fid, 'Q =\n'); write_matrix(fid, results(i).Q);
248     fprintf(fid, 'R =\n'); write_matrix(fid, results(i).R);
249     fprintf(fid, 'P =\n'); write_matrix(fid, results(i).P);
250     fprintf(fid, 'L =\n'); write_matrix(fid, results(i).L);
251     fprintf(fid, 'eig(A+L*C) =\n'); write_vector_complex(fid,
    results(i).eigAerr);
252     fprintf(fid, 'max ||e(t)|| = %.12g\n', results(i).eNormMax);
253     fprintf(fid, '||e(tEnd)|| = %.12g\n', results(i).eNormEnd);
254     end
255     fclose(fid);
256 end
257
258 function write_matrix(fid, M)
259     for i = 1:size(M, 1)
260         fprintf(fid, ' ');
261         fprintf(fid, '%.12g ', M(i, :));
262         fprintf(fid, '\n');
263     end
264 end
265
266 function write_vector_complex(fid, v)
267     for i = 1:numel(v)
268         fprintf(fid, '%+.12g%+.12gi\n', real(v(i)), imag(v(i)));
269     end
270 end
271
272 function export_model_scheme(modelName, outPath)
273     open_system(modelName);
274     try
275         print(['-s' modelName], '-dsvg', outPath);
276     catch
277         saveas(get_param(modelName, 'Handle'), outPath);
278     end
279 end
280
281 function save_svg(fig, outPath)
282     set(fig, 'PaperPositionMode', 'auto');

```

```
283     print(fig, outputPath, '-dsvg');
284 end
```

MATLAB-скрипт для задания 3

```
1  clear; clc; close all;
2
3  scriptDir = fileparts(mfilename('fullpath'));
4  cd(scriptDir);
5
6  taskDir = fullfile(scriptDir, 'third_task');
7  modelFileName = 'third_model';
8  modelPath = fullfile(taskDir, [modelFileName '.slx']);
9  [~, modelName] = fileparts(modelPath);
10 outDir = fullfile(taskDir, 'outputs');
11 if ~exist(outDir, 'dir')
12     mkdir(outDir);
13 end
14
15 A = [6 0 -12 6;
16      0 6 -6 12;
17      -12 -6 6 0;
18      6 12 0 6];
19 B = [6 0;
20      12 0;
21      6 0;
22      4 0];
23 C = [-6 6 6 6;
24      3 0 0 3];
25 D = [0 2;
26      0 2];
27
28 x0 = [1; 1; 1; 1];
29 xhat0 = zeros(4, 1);
30
31 QK = eye(4);
32 RK = eye(2);
33 QL = eye(4);
```

 Matlab

```

34  RL = eye(2);
35  nu = 1;
36
37  tEnd = 10;
38  dt = 0.002;
39  t = (0:dt:tEnd).';
40
41  f = [sin(2 * t), cos(2 * t), 0.5 * sin(3 * t), 0.5 * cos(3 * t)];
42  xi = [0.2 * sin(4 * t), 0.2 * cos(4 * t)];
43  f_ts = timeseries(f, t);
44  xi_ts = timeseries(xi, t);
45
46  U = ctrb(A, B);
47  O = obsv(A, C);
48  eigA = eig(A);
49  hautusCtrbRanks = zeros(numel(eigA), 1);
50  hautusObsvRanks = zeros(numel(eigA), 1);
51  for i = 1:numel(eigA)
52      hautusCtrbRanks(i) = rank([eigA(i) * eye(4) - A, B]);
53      hautusObsvRanks(i) = rank([eigA(i) * eye(4) - A; C]);
54  end
55
56  PK = care(A, B, QK, RK);
57  K = -RK \ (B.' * PK);
58  Acl = A + B * K;
59
60  PL = care(A.', C.', QL, RL);
61  L = -(PL * C.') / RL;
62  Aerr = A + L * C;
63
64  if bdIsLoaded(modelName)
65      close_system(modelName, 0);
66  end
67  load_system(modelPath);
68  set_param(modelName, 'StopTime', 'tEnd');
69  set_param(modelName, 'SolverType', 'Variable-step', 'Solver', 'ode45',
    'RelTol', '1e-7', 'AbsTol', '1e-9');
70  check_third_model(modelName);

```

```

71 export_model_scheme(modelName, fullfile(outDir,
    'third_task_scheme.svg'));
72
73 assignin('base', 'A', A);
74 assignin('base', 'B', B);
75 assignin('base', 'C', C);
76 assignin('base', 'D', D);
77 assignin('base', 'K', K);
78 assignin('base', 'L', L);
79 assignin('base', 'x0', x0);
80 assignin('base', 'xhat0', xhat0);
81 assignin('base', 'f_ts', f_ts);
82 assignin('base', 'xi_ts', xi_ts);
83 assignin('base', 'tEnd', tEnd);
84
85 simOut = sim(modelName, 'SrcWorkspace', 'base', 'StopTime',
    num2str(tEnd));
86 data = collect_data(simOut, K);
87
88 save_plots(outDir, data);
89 write_summary(fullfile(outDir, 'task3_summary.txt'), A, B, C, D, x0,
    xhat0, ...
90     QK, RK, QL, RL, nu, U, 0, eigA, hautusCtrbRanks, hautusObsvRanks, ...
91     PK, K, Acl, PL, L, Aerr, data);
92 save(fullfile(outDir, 'task3_results.mat'), 'A', 'B', 'C', 'D', 'x0',
    'xhat0', ...
93     'QK', 'RK', 'QL', 'RL', 'nu', 'U', '0', 'eigA',
    'hautusCtrbRanks', ...
94     'hautusObsvRanks', 'PK', 'K', 'Acl', 'PL', 'L', 'Aerr', 'data');
95
96 bdclose(modelName);
97
98 function check_third_model(modelName)
99     checks = {
100         'FromWorkspace', 'VariableName', 'f_ts';
101         'FromWorkspace', 'VariableName', 'xi_ts';
102         'Gain', 'Gain', 'A';
103         'Gain', 'Gain', 'B';

```



```

104     'Gain', 'Gain', 'C';
105     'Gain', 'Gain', 'D';
106     'Gain', 'Gain', 'K';
107     'Gain', 'Gain', 'L';
108     'Integrator', 'InitialCondition', 'x0';
109     'Integrator', 'InitialCondition', 'xhat0';
110     'ToWorkspace', 'VariableName', 'x_sim';
111     'ToWorkspace', 'VariableName', 'xhat_sim';
112     'ToWorkspace', 'VariableName', 'e_sim';
113     'ToWorkspace', 'VariableName', 'y_sim';
114 };
115 for i = 1:size(checks, 1)
116     blocks = find_system(modelName, 'SearchDepth', 1, ...
117         'BlockType', checks{i, 1}, checks{i, 2}, checks{i, 3});
118     if isempty(blocks)
119         error('В модели отсутствует блок типа "%s" с параметром
120             "%s=%s".', ...
121             checks{i, 1}, checks{i, 2}, checks{i, 3});
122     end
123 end
124
125 function data = collect_data(simOut, K)
126     data.t = simOut.tout(:);
127     data.x = normalize_signal(read_workspace_signal(simOut, 'x_sim'),
128         numel(data.t));
129     data.xhat = normalize_signal(read_workspace_signal(simOut,
130         'xhat_sim'), numel(data.t));
131     data.e = normalize_signal(read_workspace_signal(simOut, 'e_sim'),
132         numel(data.t));
133     data.y = normalize_signal(read_workspace_signal(simOut, 'y_sim'),
134         numel(data.t));
135     data.u = (K * data.xhat.').';
136     data.eNormMax = max(vecnorm(data.e, 2, 2));
137     data.eNormEnd = norm(data.e(end, :));
138     data.uNormMax = max(vecnorm(data.u, 2, 2));
139 end
140

```

```

137 function y = read_workspace_signal(simOut, name)
138     raw = simOut.get(name);
139     if isa(raw, 'timeseries')
140         y = raw.Data;
141     elseif isstruct(raw) && isfield(raw, 'signals')
142         y = raw.signals.values;
143     else
144         y = raw;
145     end
146     y = squeeze(y);
147 end
148
149 function y = normalize_signal(y, n)
150     if isvector(y)
151         y = y(:);
152     end
153     if size(y, 1) ~= n
154         y = y.';
155     end
156 end
157
158 function save_plots(outDir, data)
159     fig = figure('Visible', 'off', 'Color', 'w');
160     set(fig, 'Position', [100, 100, 1100, 900]);
161     tiledlayout(4, 1, 'TileSpacing', 'compact', 'Padding', 'compact');
162     for k = 1:4
163         nexttile;
164         plot(data.t, data.x(:, k), 'LineWidth', 2.0); hold on;
165         plot(data.t, data.xhat(:, k), '--', 'LineWidth', 2.0);
166         grid on;
167         ylabel(sprintf('x_%d', k), 'FontSize', 13);
168         set(gca, 'FontSize', 12);
169         if k == 1
170             title('Состояние и оценка состояния в LQG-системе',
171                 'FontSize', 16);
171         end
172         if k == 4
173             xlabel('t, c', 'FontSize', 14);

```

```

174         end
175     end
176     lgd = legend('x_i(t)', 'hat{x}_i(t)', 'Location', 'southoutside', ...
177         'Orientation', 'horizontal', 'FontSize', 13);
178     lgd.Layout.Tile = 'south';
179     save_svg(fig, fullfile(outDir, 'x_xhat.svg'));
180     close(fig);
181
182     fig = figure('Visible', 'off', 'Color', 'w');
183     set(fig, 'Position', [100, 100, 1000, 720]);
184     plot(data.t, data.e, 'LineWidth', 2.0);
185     grid on;
186     xlabel('t, c', 'FontSize', 16);
187     ylabel('e_i(t)', 'FontSize', 16);
188     title('Невязка наблюдателя e(t)=x(t)-hat{x}(t)', 'FontSize', 18);
189     lgd = legend('e_1', 'e_2', 'e_3', 'e_4', 'Location',
190         'southoutside', ...
191         'Orientation', 'horizontal', 'FontSize', 14);
192     lgd.Box = 'off';
193     set(gca, 'FontSize', 14);
194     save_svg(fig, fullfile(outDir, 'e.svg'));
195     close(fig);
196
197     fig = figure('Visible', 'off', 'Color', 'w');
198     set(fig, 'Position', [100, 100, 1000, 720]);
199     plot(data.t, data.u(:, 1), 'LineWidth', 2.2); hold on;
200     plot(data.t, data.u(:, 2), '--', 'LineWidth', 2.2);
201     grid on;
202     xlabel('t, c', 'FontSize', 16);
203     ylabel('u_i(t)', 'FontSize', 16);
204     title('Управление u(t)=K hat{x}(t)', 'FontSize', 18);
205     lgd = legend('u_1(t)', 'u_2(t)', 'Location', 'southoutside', ...
206         'Orientation', 'horizontal', 'FontSize', 14);
207     lgd.Box = 'off';
208     set(gca, 'FontSize', 14);
209     save_svg(fig, fullfile(outDir, 'u.svg'));
210     close(fig);

```

```

211 end
212
213 function write_summary(filePath, A, B, C, D, x0, xhat0, QK, RK, QL, RL,
    nu, ...
214     U, 0, eigA, hautusCtrbRanks, hautusObsvRanks, PK, K, Acl, PL, L,
    Aerr, data)
215     fid = fopen(filePath, 'w');
216     fprintf(fid, 'A =\n'); write_matrix(fid, A);
217     fprintf(fid, 'B =\n'); write_matrix(fid, B);
218     fprintf(fid, 'C =\n'); write_matrix(fid, C);
219     fprintf(fid, 'D =\n'); write_matrix(fid, D);
220     fprintf(fid, 'x0 =\n'); write_matrix(fid, x0);
221     fprintf(fid, 'xhat0 =\n'); write_matrix(fid, xhat0);
222     fprintf(fid, 'QK =\n'); write_matrix(fid, QK);
223     fprintf(fid, 'RK =\n'); write_matrix(fid, RK);
224     fprintf(fid, 'QL =\n'); write_matrix(fid, QL);
225     fprintf(fid, 'RL =\n'); write_matrix(fid, RL);
226     fprintf(fid, 'nu = %.12g\n', nu);
227     fprintf(fid, '\nrank(ctrb(A,B)) = %d\n', rank(U));
228     fprintf(fid, 'rank(obsv(A,C)) = %d\n', rank(0));
229     fprintf(fid, 'eig(A) =\n'); write_vector_complex(fid, eigA);
230     fprintf(fid, 'Hautus controllability ranks:\n');
231     for i = 1:numel(eigA)
232         fprintf(fid, 'lambda = %.12g%.12gi, rank = %d\n',
            real(eigA(i)), imag(eigA(i)), hautusCtrbRanks(i));
233     end
234     fprintf(fid, 'Hautus observability ranks:\n');
235     for i = 1:numel(eigA)
236         fprintf(fid, 'lambda = %.12g%.12gi, rank = %d\n',
            real(eigA(i)), imag(eigA(i)), hautusObsvRanks(i));
237     end
238     fprintf(fid, '\nPK =\n'); write_matrix(fid, PK);
239     fprintf(fid, 'K =\n'); write_matrix(fid, K);
240     fprintf(fid, 'eig(A+B*K) =\n'); write_vector_complex(fid, eig(Acl));
241     fprintf(fid, '\nPL =\n'); write_matrix(fid, PL);
242     fprintf(fid, 'L =\n'); write_matrix(fid, L);
243     fprintf(fid, 'eig(A+L*C) =\n'); write_vector_complex(fid, eig(Aerr));
244     fprintf(fid, '\nmax ||e(t)|| = %.12g\n', data.eNormMax);

```

```

245     fprintf(fid, '||e(tEnd)|| = %.12g\n', data.eNormEnd);
246     fprintf(fid, 'max ||u(t)|| = %.12g\n', data.uNormMax);
247     fclose(fid);
248 end
249
250 function write_matrix(fid, M)
251     for i = 1:size(M, 1)
252         fprintf(fid, ' ');
253         fprintf(fid, '%.12g ', M(i, :));
254         fprintf(fid, '\n');
255     end
256 end
257
258 function write_vector_complex(fid, v)
259     for i = 1:numel(v)
260         fprintf(fid, '%+.12g%+.12gi\n', real(v(i)), imag(v(i)));
261     end
262 end
263
264 function export_model_scheme(modelName, outPath)
265     open_system(modelName);
266     print(['-s' modelName], '-dsvg', outPath);
267 end
268
269 function save_svg(fig, outPath)
270     set(fig, 'PaperPositionMode', 'auto');
271     print(fig, outPath, '-dsvg');
272 end

```